PROGRAMMATION SYSTEME sous LINUX

Ecole Centrale de Lille,
CS 20048, F59651 Villeneuve d'Ascq Cedex
Tél : (33)(0)3 20 33 54 56, (33)(0)6 80 08 48 64
E-mail : samir.elkhattabi@centralelille.fr

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Propos du support

Ici, on s'intéresse uniquement aux architectures à base de **micro-processeur** (les architectures à base de microcontrôleur ne sont pas étudiées).

Ainsi, le système électronique peut être qualifié de calculateur et donc doté d'un **système d'exploitation multitâches** et si nécessaire **temps réel**.

Ici, on s'intéresse uniquement à la **programmation système** en vue de la conception d'applications concurrentes, portables & robustes (embarquées et le plus souvent temps-réel) :

- la génération d'une solution système à embarquer fait l'objet d'un autre support.

Au vu des contraintes matérielles et de l'environnement de ces dispositifs, on utilisera une plateforme **UNIX** et une implémentation **POSIX** pour garantir/faciliter la **réplicabilité** du système d'exploitation et la **portabilité** des applications.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Système Temps-réel & embarqué

Un système **embarqué**, composé d'une partie **électronique** (matériel) et d'une partie **informatique**, est généralement dédié à une tâche spécialisée et ce de manière autonome.

Il est généralement **enfoui** :

- L'encombrement est réduit et donc les ressources matérielles le sont aussi,
- la consommation énergétique doit être réduite

Il est souvent **temps-réel** (répond à des contraintes temporelles).

Et il doit être **sûr de fonctionnement**.

Les transparents pour le TR ont un fond quadrillé.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

GÉNÉRALTIÉS SUR LES SYSTÈMES D'EXPLOITATION --- UNIX / LINUX

Histoire des systèmes d'exploitation

Système d'exploitation : rôle, modèle en couches

UNIX et ses branches

POSIX & Programmation système

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

Historique des OS – les débuts

1° génération (1945 – 1955) :

- **Von Newman** propose la première machine considérée comme un ordinateur.
- Tube à vide et tableaux d'interrupteurs,
- **Pas de système d'exploitation** : langage machine basé sur le **câblage** des entrées et sorties,
- Les programmeurs n'ont pas accès à la machine (**cartes perforées**),
- **ENIAC** (université de Pennsylvanie).

2° génération (1950 – 1965) :

- Apparition du **transistor** en 1950.
- Premier ordinateur commercialisé : **IBM 7094** doté du premier système **IBM SYS** --- création d'un chargeur (**loader**) ---.
- Les programmeurs n'ont toujours pas accès à la machine :
  - Programme utilisateur encodé sur cartes perforées en **Assembleur/FORTRAN**,
  - Regroupement des programmes par lot sur **bande magnétique** : **batch**,
  - Résultats sur **imprimante**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Historique des OS – la naissance du multitâches

3° génération (1965 – 1980) :

- Apparition du circuit intégré.
- IBM présente son System/360 :
  - Plusieurs séries seront produites (370, 4300, 3080, 3090),
  - Performances variables et compatibilité.
- DIGITAL propose le premier mini-ordinateur le DEC PDP-1.

Le temps inhérent aux E/S était très important :

- À peine 20% du temps de traitement était réellement affecté au calcul,
- Il fallait donc occuper le processeur pendant les opérations d'E/S,
- Apparition alors de la **multiprogrammation** basée sur le **partitionnement** de la mémoire. On distingue alors deux types de **tâches** programmables : celles liées aux E/S et celles liées au calcul.

Simultaneous Peripheral Operation On Line (SPOOL) :

- Plusieurs tâches sont chargées en mémoire,
- Le système **commute** entre les tâches sur **blocage** d'E/S.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Historique – Temps partagé & naissance d'UNICS

Darthmouth College et le MIT inventent le temps partagé :

- Introduction des **terminaux** : accès direct pour les utilisateurs,
- Le système de Darthmouth ne permettait de faire que du **BASIC**,
- **CTTS** (Compatible Time Sharing System) eut un franc succès auprès des scientifiques.

MIT, Bell et General Electric s'associent pour créer une deuxième génération de systèmes d'exploitation, pour rendre le développement système **indépendant** du matériel (abstraction) :

- **MULT**iplexed **I**nformation and **C**omputing **S**ervice : système multi-utilisateurs,
- MULTICS était écrit en **PL/I** et le compilateur est arrivé tardivement : échec mais de nombreuses innovations,
- MIT arrivera à produire une version opérationnelle en 1980, commercialisé par HP qui a racheté les activités informatiques de GE.
- En 1969, Ken Thompson (ancien programmeur de Bell) eut l'idée de réécrire une version simplifiée, mono-utilisateur de MULTICS sur un PDP-7 appelée **UNICS** rebaptisée **UNIX (cf. plus loin)**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Historique – 4° génération (Depuis 1980) (1)

Apparition des **circuits LSI** (Large Scale Integration circuits).

Intel sort le 8080 (premier processeur à **8 bits**), **Gary Kildall** conçoit un système d'exploitation orienté disque :

- **CP/M** (Control Program for Micro-computers) créant ainsi le **premier micro-ordinateur** équipé d'un disque 8 pouces,
- Kildall fonde Digital Research pour commercialiser le CP/M.

En 1980, IBM crée l'IBM PC.

- Après refus de Kildall de concevoir un système pour son PC, IBM se tourne vers **Bill Gates** qui achète les droits du DOS (Disk Operating System) proposé par Seattle Computer Products (50000 $ selon les rumeurs).
- Gates fonde Microsoft et engage Tim Paterson (créateur de DOS) et après quelques modifications souhaitées par IBM, la première version de MS-DOS était née.

En 1983, avec l'apparition du PC/AT (Intel 80286), MS-DOS était bien implanté alors que CP/M vivait ses derniers moments.

- La version initiale était très primitive et se voit enrichir avec des mécanismes empruntés à UNIX,
- Plusieurs versions vont voir le jour accompagnant plusieurs générations de processeurs Intel (80386, 80486).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Historique – 4° génération (Depuis 1980) (2)

S. Jobs d'Apple invente l'Apple :

- Premier ordinateur avec un OS à interface graphique,
- Arrivée du Macintosh : un grand succès.

Microsoft contrattaque timidement en lançant plusieurs versions de Windows au-dessus du DOS :

- Version 1 & 2 : interface simple, échec,
- Version 3.1 : enfin une version qui a marché,
- Windows 95/98 : 16 bits, 1° versions Windows indépendantes du DOS.

Microsoft propose enfin un système totalement réécrit par David Cutler (ancien concepteur de VMS) et son équipe :

- Windows NT : New Technology, 32 bits,
- Une certaine compatibilité avec les différentes versions du DOS,
- Beaucoup de concepts empruntés à VMS,
- NT 4 est la première version considérée comme stable,
- En 1999, arrivée de NT 5 (baptisé 2000) pour unifier Win 98 et NT.
- En 2003, arrivée de Windows XP et en 2007, arrivée de Windows Vista, **...**

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Historique – 4° génération (Depuis 1980) (3)

UNIX se dote d'un système de fenêtrage appelé **X Window** autour du standard **X11** produit par le MIT :

- Gestionnaire de fenêtres avec une IHM adaptable,
- Exemples d'IHM : Motif, OpenLook, DCE.

Plusieurs constructeurs proposent leur propre UNIX (architecture RISC) : IBM, Sun, Digital, Bull, HP, ...

Milieu des années 80, la croissance des réseaux a amené le développement des systèmes d'exploitation **distribués** :

- Système classique doté en plus des primitives de communication réseau adaptées.

Linus Torvalds propose une version d'UNIX pour PC (architecture Intel) en 1991 appelé **LINUX**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Historique – Résumé

Serial Processing (~1950)

- Lancement manuel d'un nouveau programme à la terminaison du précèdent,

Simple Batch Systems (~1960)

- File d'attente de programmes,

Multiprogrammed Batch Systems (~1970)

- Exploiter le matériel en alternant les processus,

Time Sharing (~1980)

- Alternance automatique entre processus,
- Interactivité,

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Historique – quelques chiffres (1)

	1ère	2ème	3ème	4ème
Nom de la machine	L'ENIAC	PDP-1	PDP-8/1	LSI-11
Composants	Tube électronique	Transistor	Circuit intégré	Microprocesseur
Année	1950	1960	1965	1976
Encombrement	Un bâtiment	Une armoire	Un rack	Une carte
Dimensions	15*15*6 m	2.4*0.75*1.8 m	60*60*60 cm	22*25*1.2 cm
Consommation (watts)	150 000	2 500	250	50
Nombre d'accès mémoire par seconde	80 000	200 000	600 000	900 000
Prix ($)	?	120 000	10 000	600

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Historique – quelques chiffres (2)

1964, OS/360 IBM :

- 1 Million d'instructions

1975, Multics et VMS de DEC :

- 1 Million d'instructions

1981, DOS : 13 Ko

1987, DOS : 78 Ko

1988,

- Unix BSD 4.2 : 500 000 instructions
- OS/VS IBM : 5 Millions d'instructions
- Multics et MVS : 20 Millions d'instructions

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Calculateur & Système d'exploitation

Tout calculateur est composé d'un **processeur**, d'une **mémoire** centrale et de **périphériques** tels que les disques, les interfaces d'interaction, des interfaces réseau et d'autres périphériques d'E/S.

Un système d'exploitation (OS pour Operating System) est un ensemble de programmes qui **prend en charge** tous ces composants matériels, qui les **utilise correctement** et ce de manière optimale en fournissant à l'utilisateur des interfaces simplifiées :

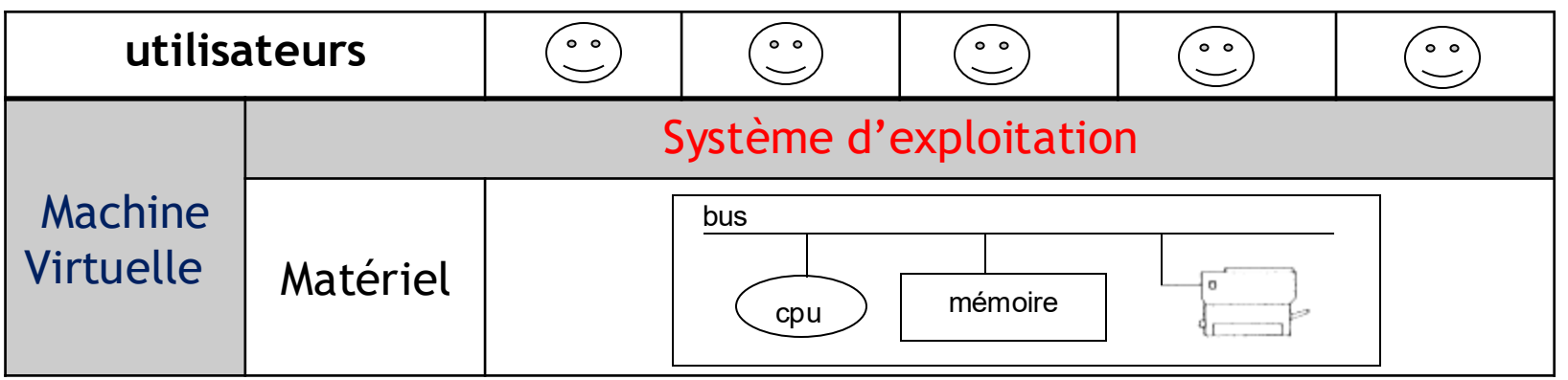
- Interface d'**interaction** : utilisateur/administrateur, environnement texte et/ou graphique,
- Interface de **programmation** : Librairies d'appels système,
- Interface de **communication** : échange de données à travers un réseau,

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Fonctions d'un OS

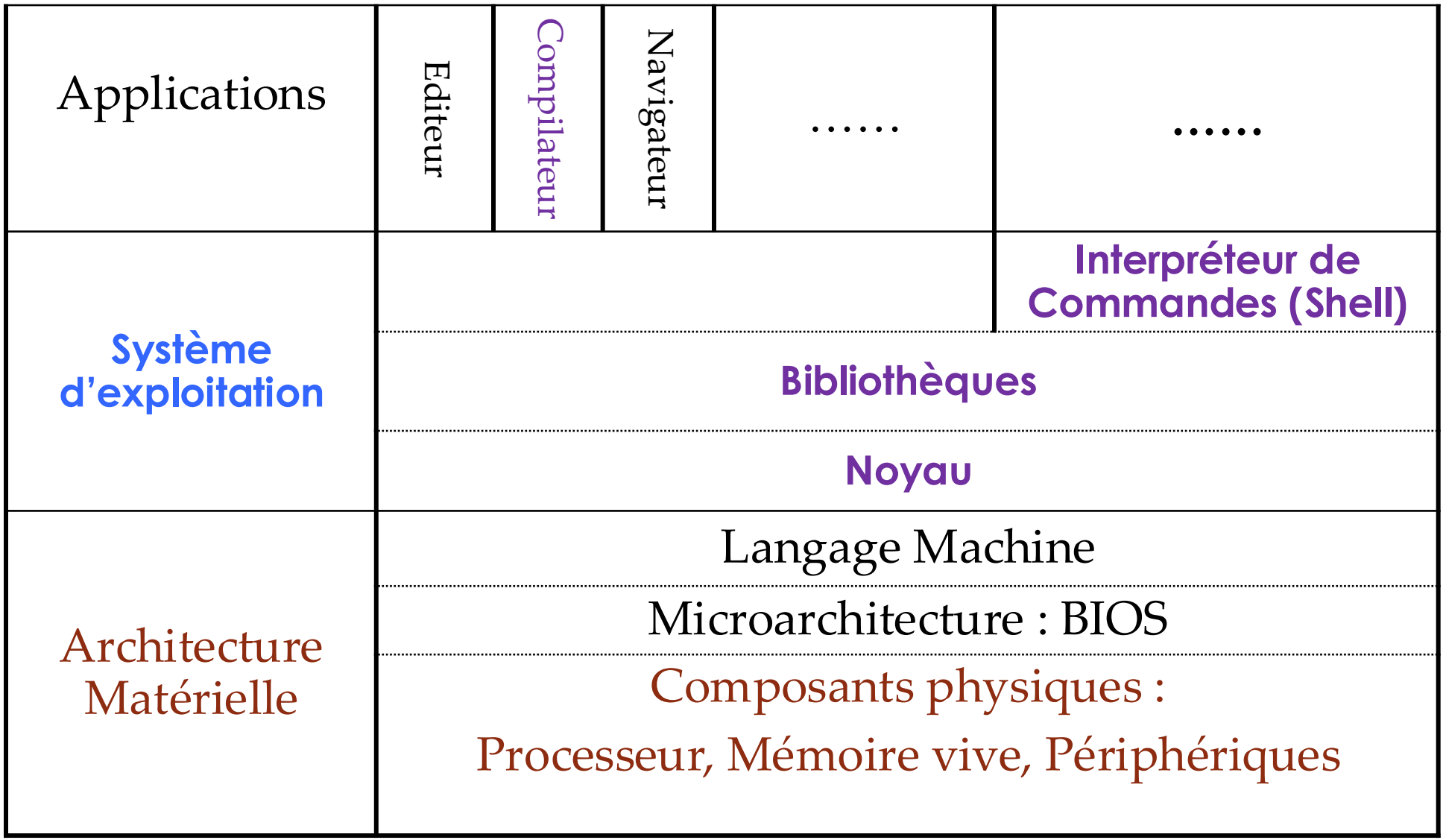
Le système d'exploitation est un ensemble de programmes qui réalise l'interface entre le matériel de l'ordinateur et les utilisateurs. Il a deux objectifs principaux :

- Construction au-dessus du matériel d'une machine virtuelle avec un **langage étendu** plus facile d'emploi et conviviale,
- Prise en charge de la **gestion** de plus en plus complexe des ressources et **partage** de celles-ci entre des tâches et/ou des utilisateurs.



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Système Informatique (1)



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Système Informatique (2) – Matériel

Le plus bas niveau est du ressort de l'électronique, il est composé de circuits intégrés, de connectique, d'alimentation en tension, …

La **micro-architecture** regroupe les composants en unités fonctionnelles comme la CPU (Central Process Unit) qui exécute un programme chargé à une adresse mémoire donnée.

Le traitement effectué par la CPU, est contrôlé par un **microprogramme** ou par des circuits matériels.

Les instructions sont exécutées en un ou plusieurs **cycles d'horloge** et sont écrites en assembleur : **Instruction Set Architecture** (ISA) appelé également langage machine.

Le langage machine est un jeu d'instructions assez réduit : de 50 à 300 instructions. Il s'agit principalement de réaliser :

- des opérations arithmétiques et logiques,
- des opérations de déplacement de données (lecture/écriture).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Système informatique (3) – Noyau …

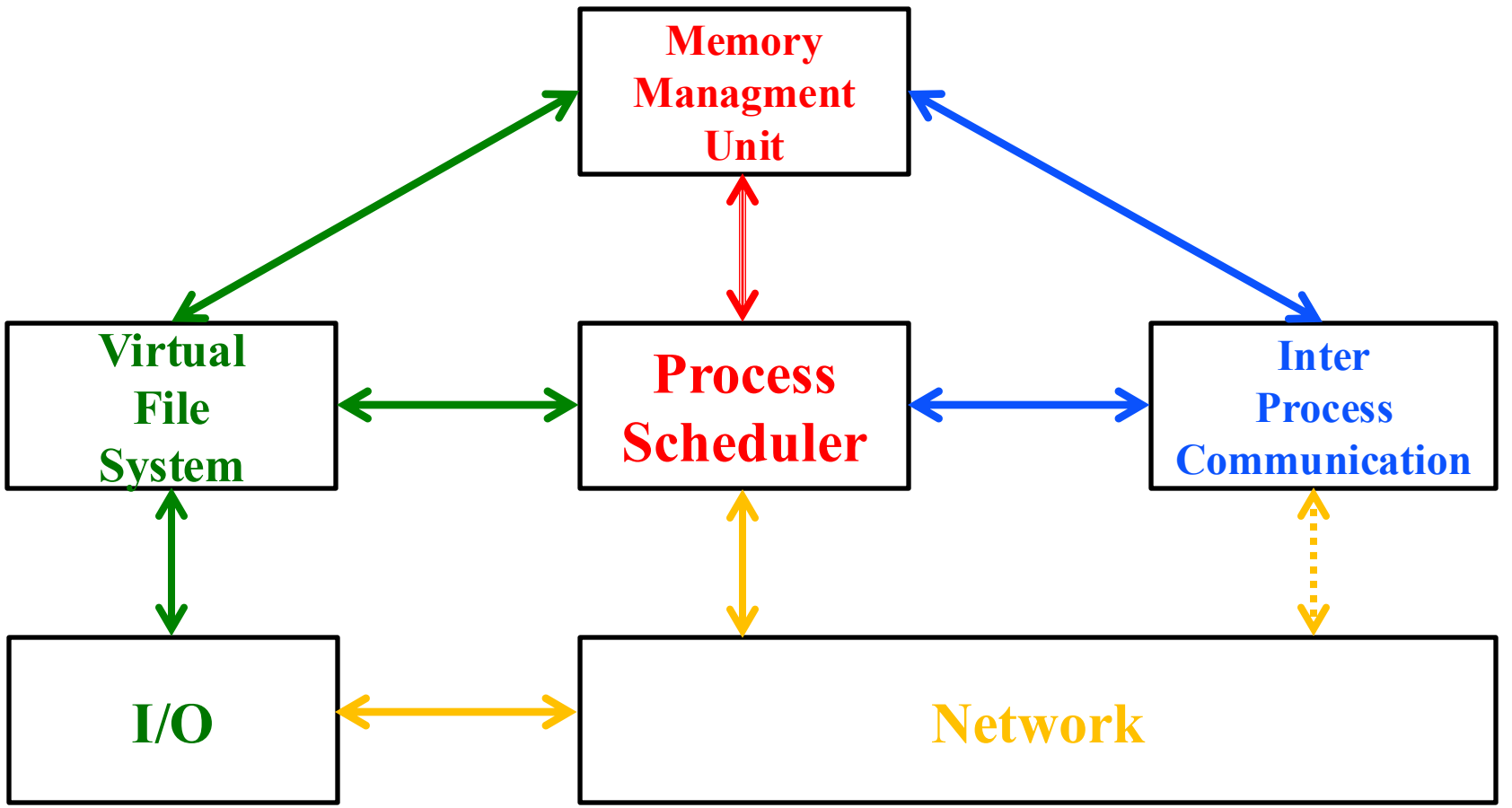
Le **noyau** est la constituant principal d'un OS. Les composantes d'un noyau s'exécutent dans l'**espace noyau** afin répond à plusieurs besoins :

- Contrôler le « **multitâche** » par un **ordonnanceur**.
- Gérer la mémoire vive par un gestionnaire mémoire dit MMU (Memory Management Unit).
- Gérer les ressources matérielles par des **pilotes** dits « matériel ».
- Mettre en en place des pilotes dits « logiciel » : TCP/IP, VFS, …
- Partager les ressources aussi bien matérielles que logicielles en les protégeant par des mécanismes de **verrous**.
- Contrôler les **droits d'accès**.

Pour un noyau temps-réel, on ajoutera la propriété de **préemption**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

…Système informatique (3) – Noyau



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Système Informatique (4) – Bibliothèques

Les périphériques sont contrôlés par des **registres de périphérique**. On peut demander par exemple :

- une lecture à un disque,
- en chargeant son adresse avec une adresse mémoire, un nombre d'octets, et le type de l'opération (lire/écrire) dans son registre.
- d'autres paramètres sont généralement nécessaires.
- un mot d'état est renvoyé à la fin d'opération.

Les bibliothèques d'un OS permettent de **masquer** cette complexité en fournissant au développeur un jeu d'instructions plus évolué et plus pratique (**bibliothèques système**). Pour notre exemple, il s'agira de lire des blocs au lieu de se préoccuper du déplacement des têtes de lecture, de la gestion de la latence, …

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Système Informatique (5)

Au-dessus de l'OS, on trouve les interfaces d'interaction (**Shell** et/ou système de fenêtrage) et les interfaces de programmation (**compilateurs**) qui permettent de créer des applications utilisateurs qui masquent à leur tour les langages de programmation.

Les applications s'exécutent dans l'**espace utilisateur**.

L'OS est un programme qui fonctionne en **mode noyau** : l'OS est **protégé** contre les modifications de l'utilisateur par des mécanismes de **protection matérielles**.

Un utilisateur peut écrire son propre compilateur s'il le souhaite par-contre il ne peut écrire son propre gestionnaire d'interruptions qui est protégé contre toute modification..

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

FAMILLE UNIX

UNICS / UNIX / C

UNIX SysV (AT&T) & UNIX BSD

LINUX --- UNIX GNU (manquant)

POSIX IEEE : interface de programmation système

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

UNICS(X)

En 1969, les laboratoires Bell se retire du projet **MULTICS** et Ken Thompson eut l'idée de réécrire une version simplifiée de MULTICS sur un **mini**-ordinateur de l'époque (un PDP-7) :

- Brian Kernighan l'appela par dérision **UNICS**.

Denis Ritchie rejoint le projet avec son département :

- Portage d'**UNIX** sur un PDP-11/20, **PDP-11/45**, **PDP-11/70** (machines largement utilisées).
- Dispositifs matériels de protection mémoire : plusieurs utilisateurs simultanés.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

UNIX & C

Thompson décide de développer un langage de haut niveau pour faciliter la réécriture du système, qu'il a appelé **B** (version simplifiée de **CPL**) :

- **Manque de structures.**

Ritchie développe alors le langage **C**.

Thompson et Ritchie ont réécrit UNIX en C : Publication décrivant **UNIX** en 1974 (ils obtiennent en 1984 le prix Turing de l'ACM).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Expansion de UNIX

Bell distribue des licences aux universitaires moyennant une modique contrepartie :

- Les universitaires étaient équipés de PDP mais avec de **mauvais** systèmes d'exploitation.
- UNIX offrait un environnement **ouvert** avec un source **modifiable**.

Les séminaires de recherche et de développement sur UNIX se multiplièrent :

- **Amélioration** du système.
- **Diffusion** rapide de nouvelles idées.
- Version 6 (6° édition du manuel) puis (quelques années plus tard) version 7 : **sysV Version 7**

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Les branches UNIX – AT&T et BSD

En 1984, AT&T est libéralisée et commercialise sa première version d'UNIX appelée System III :

- Un an plus tard, **System V** était disponible (System IV est un mystère).
- Release 2, Release 3, Release 4 : complexité croissante.

L'université de Berkeley aidée par les subventions de la DARPA a apporté de nombreuses modifications :

- Une première version fut appelée 1**BSD**, suivie de 2BSD sur PDP-11.
- **4BSD** développé pour le VAX a fait le succès de cette version à l'inverse de AT&T qui offrait une version 7 grossie.

Améliorations :

- Mémoire virtuelle & **pagination**.
- **Optimisation** du **système de fichiers**.
- **Fiabilisation des signaux**.
- **Communication** entre machines : **TCP/IP** s'impose comme standard (#OSI).
- Utilitaires & compilateurs : vi, csh, Lisp, Pascal.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Portabilité d'UNIX

Le premier portage vers une architecture différente (Interdata 8/32) a permis de mettre en évidence l'insuffisance des hypothèses initiales.

Exemple : un entier sur 16 bits, un pointeur est codé sur 16 bits $\Rightarrow$ taille de programme limitée à 64 K.

Une abstraction du matériel (ici, la mémoire) était nécessaire.

Depuis UNIX a été portée sur une grande variété d'ordinateurs.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Portabilité d'UNIX

Le portage nécessite de :

- Disposer d'un **compilateur** C sur la machine cible.
- Disposer des **pilotes** pour les périphériques d'E/S (C et assembleur).
- Ecrire un **gestionnaire d'interruption** en langage machine.
- Ecrire les **routines de gestion de la mémoire**.

Le portage a été facilité par l'écriture d'une version C portable par Steve Johnson.

Le portage et l'installation d'UNIX sur une nouvelle machine est devenu chose facile.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Noyau UNIX & portabilité

Le noyau UNIX est composé de deux parties essentielles.

Un noyau **dépendant** de la machine : écrit en C mais doit être réécrit en assembleur en cas de portage sur une nouvelle machine.

- gestionnaire d'interruptions,
- pilotes d'E/S,
- une partie du logiciel de gestion mémoire.

Noyau **indépendant** de la machine (il est le même sur toutes les machines) : entièrement écrit en C.

- gestion des appels systèmes,
- gestion des processus, ordonnanceur,
- pagination, swapping,
- système de fichiers, tubes,

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

POSIX & Programmation Système

Standard POSIX IEEE

Compilation

Gestion d'erreurs

Aide en ligne

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

Standard UNIX

Début 80, chaque constructeur propose sa version d'UNIX généralement basée sur **4.3BSD** et rarement sur System V R3 (malgré la publication de System V Interface Definition) avec une **incompatibilité dans le binaire** (pas de norme initiale).

Regroupement de constructeurs :

- **O**pen **S**oftware **F**undation (intégrer la norme POSIX, X11) :
  - Constitué de : IBM, Digital, HP
  - Travaux de spécifications en vue de la standardisation
  - **OSF/1 V2.0** de Digital : Digital Unix.
- **UI** (**U**nix **I**nternational) : Sun, AT&T, …
- **X/OPEN** : association européenne (Guide XPG III).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

POSIX

Norme IEEE 1003 (ensuite ISO) pour la normalisation des appels systèmes : Portable Operating System IX (pour UNIX)

- Un bon compromis : on a procédé par intersection et les différences ont été ignorées. Résultat : version 7 + gestion des signaux de BSD + gestion des terminaux.

Normes POSIX :

- POSIX 1003.1a : création de processus et exécution, gestions des I/O et fichiers
- POSIX 1003.1b : communication interprocessus, allocation du CPU, gestion de la mémoire et Asynchronisme
  - (P1003.1b : correspond à P1003.4/D14)
- POSIX 1003.1c : multithreads
  - POSIX 1003.4a/D4 : (i.e. Draft) non publié
- ….

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Pourquoi POSIX ?

Normalisé par Institute of Electrical and Electronic Engineers (**IEEE**) et **adopté** par International Organization for Standardization (**ISO**) et International Electrotechnical Commision (**IEC**)

Solution portable non seulement sur des machines UNIX mais également sous Windows NT.

POSIX SPECIFIE L'INTERFACE AVEC L'OS

MAIS N'EST PAS l'OS

POSIX N'EST PAS UNE IMPLEMENTATION

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Objectifs du cours – Programmation système

Faire une **programmation avancée** mettant en jeu :

- des **ressources à partager**
  - mémoire, CPU, I/O
- **des processus concourants & multithreading**
- de la communication et de la **synchronisation interprocessus**
  - **signaux – sémaphores – mémoire partagée –** Messages
- des algorithmes d'allocation du CPU
  - FIFO, Tourniquet

Et du TEMPS REEL :

- du synchronisme, de l'asynchronisme des E/S
- Horloges et Temporisateurs (seconde, microseconde et nanoseconde)
- ...

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

PSY – Mise en oeuvre

Comment : Mise en œuvre en **C**

- **Insertion de la librairie standard POSIX dans votre code**

  #include <unistd.h>
- Compilation :
  - **-L** pour indiquer le chemin d'accès à une librairie
  - **-l** pour inclure une librairie
  - **-lpthtread** : libraire des threads et sémaphores
  - **-lrt** : librairie temps-réel
  - **-non_shared** car pas de partage en temps réel
- Exemple :

  $ gcc -non_shared myprg.c -o myprg -L/usr/css/lib -lrt -lpthread

  $ gcc myprg.c -o myprg -lpthread

Documents :

- manuels on line

  $ man 2 appel-système

  Exemple : $ man 2 fork

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

PSY – Aide sous UNIX

Le **manuel** est divisé en **huit** sections (au moins) :

- 1 : Les commandes utilisateurs
- 2 : **Les appels système**
- 3 : **La librairie des sous-routines**
- 4 : Les formats de fichiers
- 5 : Les fichiers spéciaux
- 7 : **Les composantes système**
- 8 ou 1m : Les commandes d'administration système
- 9 : glossaire

Affichage paginé (more) des informations concernant une commande (ou toutes les commandes concernant un mot clé) :

man [**n**] <commande> n représente un numéro de section

man -k <mot clé> affiche une liste de commandes pour lesquelles le mot clé recherché apparaît dans le résumé

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

PSY – Gestion des erreurs

L'appel d'une fonction système POSIX retourne une valeur **positive** en cas de succès et **-1** en cas d'échec.

Dans ce dernier cas, la variable **errno** est positionnée avec le numéro d'erreur.

Utilisation de **perror()** :

- perror() affiche un message relatif à errno qui est utilisé comme **index** dans une **table de messages d'erreurs**.
- perror() affiche (**aussi**) le message passé en argument pour personnaliser le message d'erreur.

On utilisera donc **systématiquement** une macro-fonction pour **vérifier** tout appel système :

```
#define CHECK(sts,msg) if ((sts) == -1) {perror(msg); exit(-1); }
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

PSY – Commandes UNIX à connaître

Manipulation de fichiers :

ls, cd, pwd, mkdir, cd, cp, rm, mv, ln, more, cat, find, tee, ...

Manipulation des permissions d'accès :

chmod, chown, chgrp, umask, newgrp, groups, ...

Gestion de processus :

ps, pstree, kill, top, ...

Filtres :

grep, sort, tail, cut, paste, tee, od, tr, pr, ...

Expressions régulières :

grep, tr, sed, awk

Compilateurs & outils de développement :

make, nm, time, prof, lint, ld, ar, svn, sccs, dbx ou gdb, ...

Développement de langage (pas pour tous) :

lex/flex, yacc/bison

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

MULTITÂCHES / PROCESSUS

Multiprogrammation, Commutation de contexte, modes d'exécution, Modèle mémoire.

Ordonnancement des processus.

Processus : Création & Finalisation.

Threads : Création & Finalisation.

Multiprocesseurs & Affinité.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

Multiprogrammation

Un ordinateur peut exécuter plusieurs tâches à la fois :

- Exécution d'un programme utilisateur
- Lecture de données sur un disque
- Afficher les résultats sur un terminal
- ….

Un OS multiprogrammé **commute** d'une tâche à une autre, en exécutant chaque tâche pendant quelques dizaines ou centaines de ms (notion de **quota CPU**).

A un instant donné, une seule tâche est exécutée : **faux** ou **pseudo-parallélisme**.

Problématique : contrôler plusieurs activités en parallèle

Modèle de parallélisme avec partage de ressources communes (besoin de mécanismes de protection)

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Notion de processus

Un **processus** est une instance dynamique d'un programme (en cours d'exécution) : une **tâche** système.

L'OS caractérise un processus par une structure dite **PCB** (Process Control Block) :

- Un identifiant,
- un **espace mémoire** composé de 3 segments,
- ...

Deux familles de processus :

- **Classique/normal** : un programme donné dédié à une tâche donnée avec un **temps d'exécution fini**.
- **Démon/permanent** : un programme de **service** (lancé au démarrage du système) qui s'**exécute en permanence** jusqu'à l'arrêt du système.

  cron, lpd, sshd, tftpd, httpd, smtpd, ...

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Multitâches & commutation

Le **multitâche** sous UNIX s'exprime par l'exécution **pseudo-parallèle** de processus.

L'ordonnancement sous UNIX repose sur la notion de **thread** : un processus est formé d'au moins un thread.

A un processus correspond un espace mémoire **partagé** entre plusieurs threads et un état d'exécution par thread.

La **commutation** entre threads est assurée par l'**ordonnanceur** alors que la gestion des segments mémoires du processus est assurée par le sous-système de gestion mémoire du noyau (**MMU**).

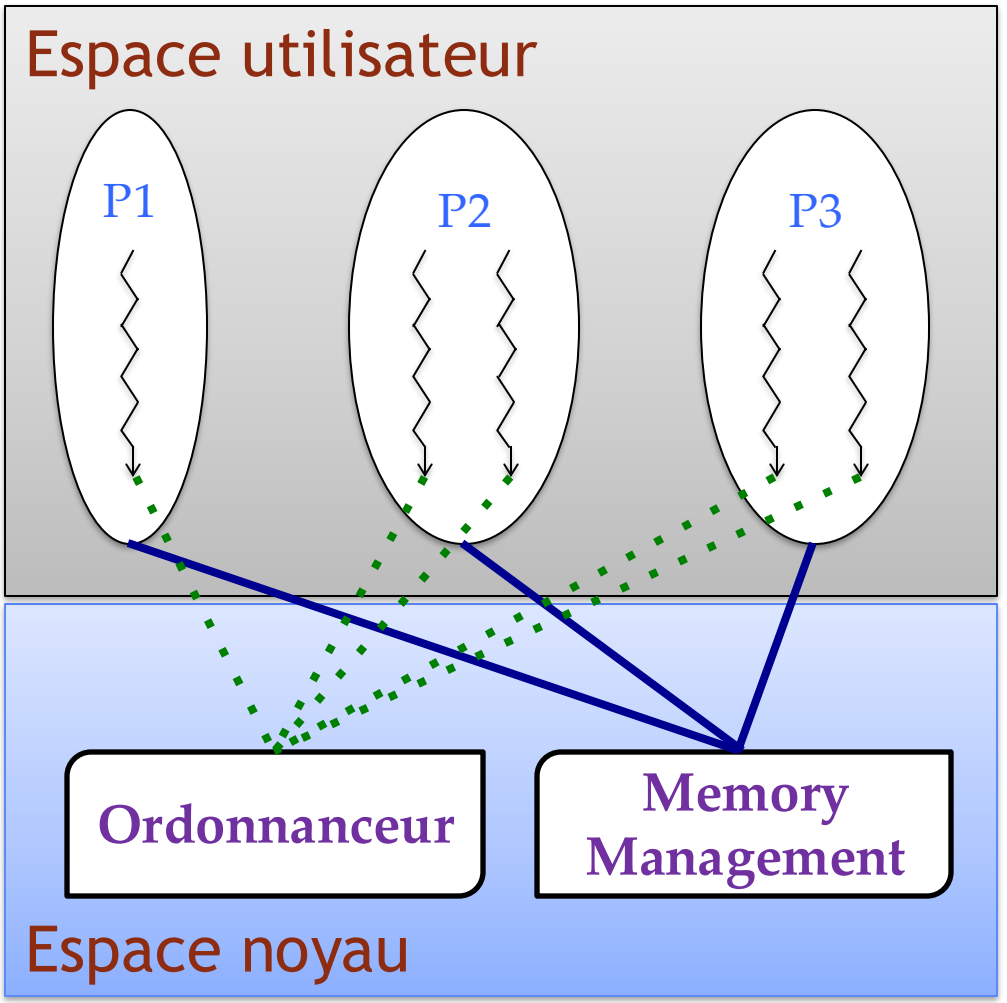
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Modes d'exécution ...

Les processus s'exécutent dans l'**espace utilisateur**.

UNIX fonctionne sur des micro-processeurs disposant d'au moins deux modes de fonctionnement :

- **mode privilégié** : accès total au jeu d'instructions, aux E/S, paramétrage de l'OS
- **mode restreint** : jeu d'instructions limité avec un accès limité à certaines E/S et un espace mémoire (accès attribué par le noyau) et sans manipulation des interruptions.



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

... Modes d'exécution

Un processus (y compris ceux exécutés avec les droits root) **bascule** entre deux modes d'exécution :

- **Noyau ou superviseur / privilégié** : un programme nécessitant l'accès à une ressource système partagée ;
- **Utilisateur / restreint** : un programme ne nécessitant aucune ressource système.

Ainsi, on dispose d'une **protection d'accès** au matériel, à la mémoire par un accès contrôlé (**uniquement accordé par le noyau**).

Toute violation de la règle provoque une exception (ouvre une trappe), en donnant généralement le contrôle au noyau pour prendre les dispositions adéquates.

Un processus **accède et utilise** les ressources du système en effectuant des **appels système**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Commutation de contexte

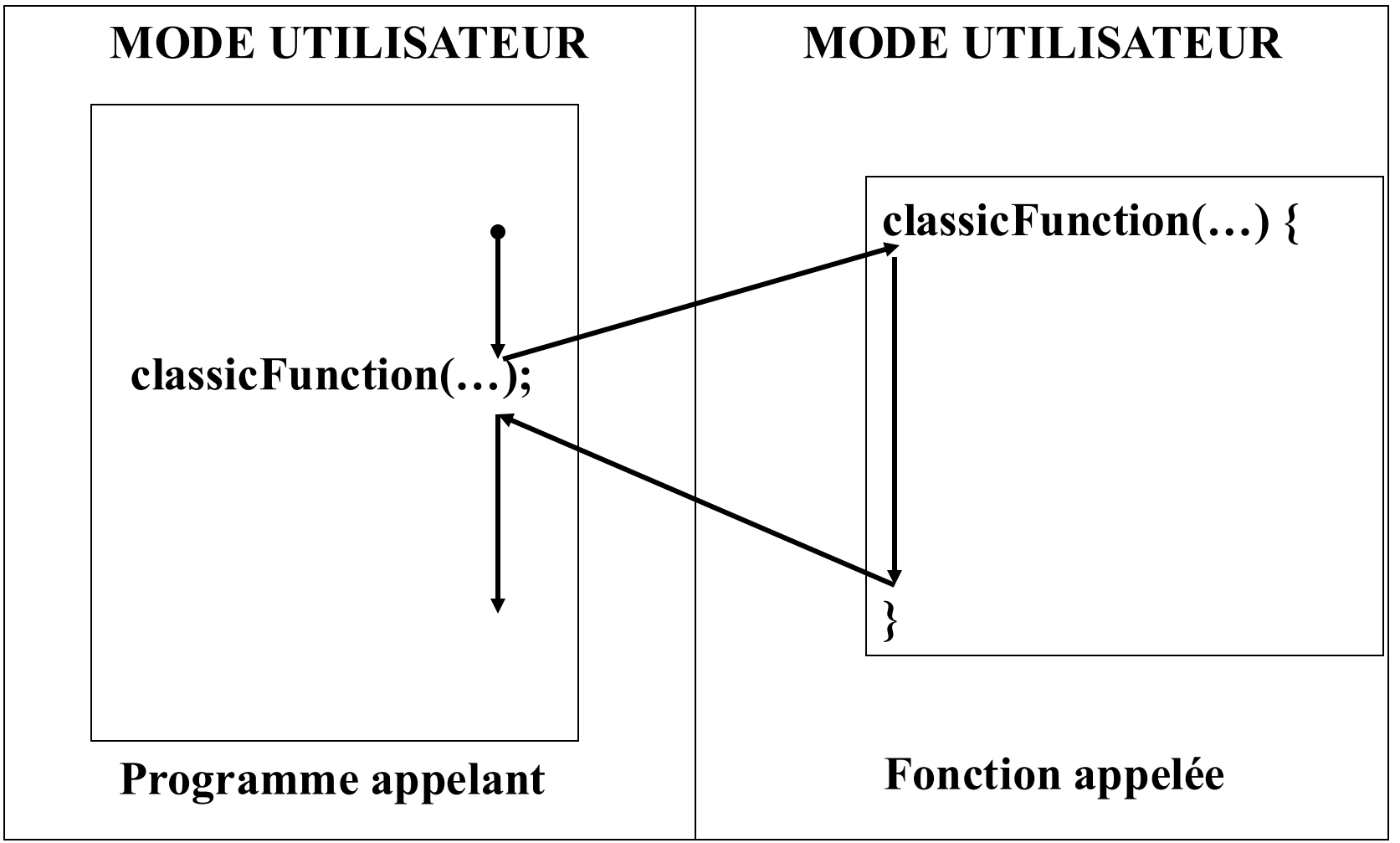
Définition : Une **commutation** de contexte caractérise la **transition** réalisée lorsqu'un programme invoque un autre programme (équivaut à l'appel d'une fonction). La fin du programme appelé rétablit le contexte initial.

A l'échelle de l'OS, cette **commutation** se produit généralement lors d'un **appel système** avec un **passage** du mode user au mode noyau.

Cette commutation s'accompagne d'une sauvegarde du contexte actuel, la création d'un nouveau contexte chargé avec l'adresse de la routine système et le passage du registre d'état en mode noyau.

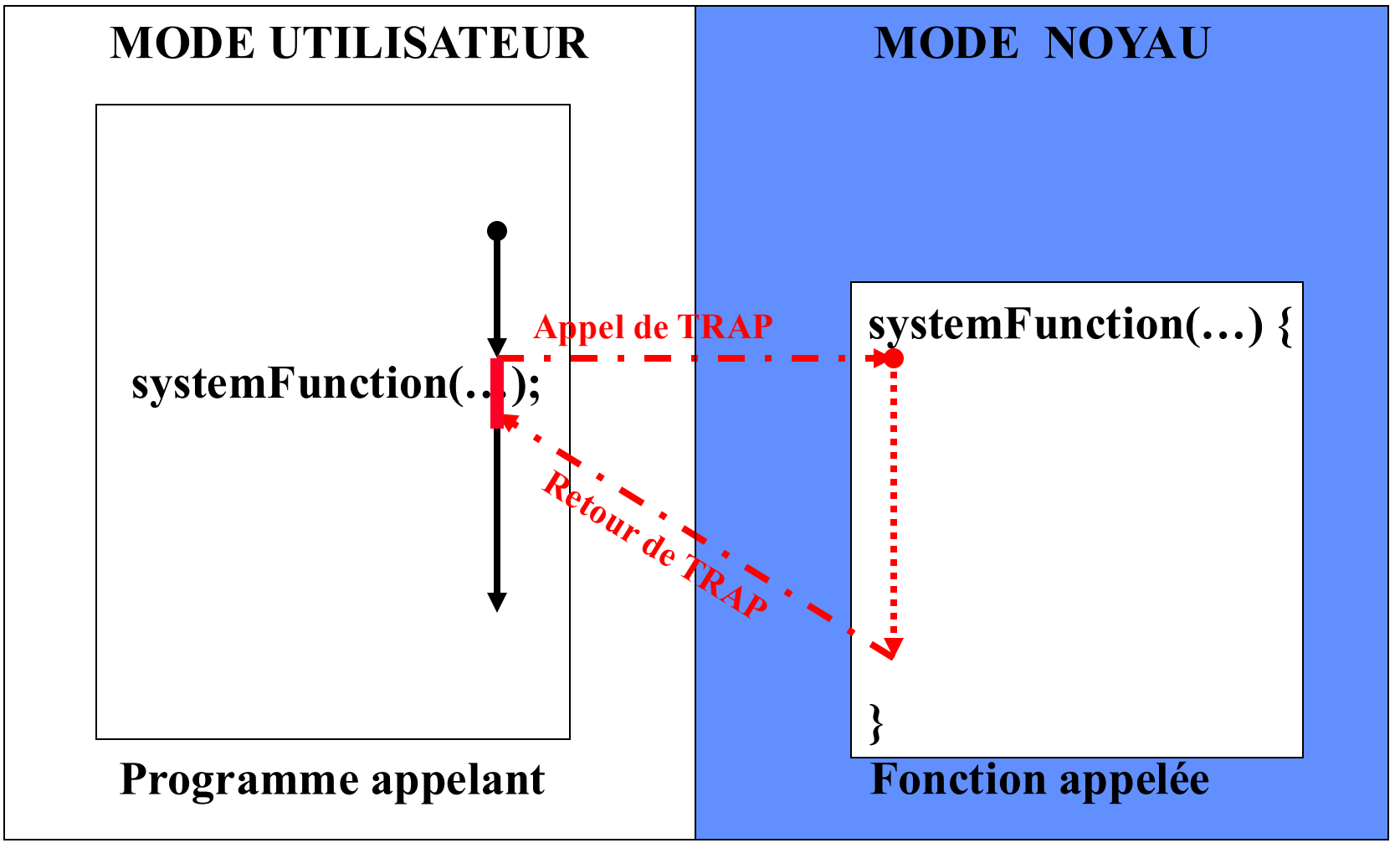
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exécution d'un appel de fonction classique



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exécution d'un appel système



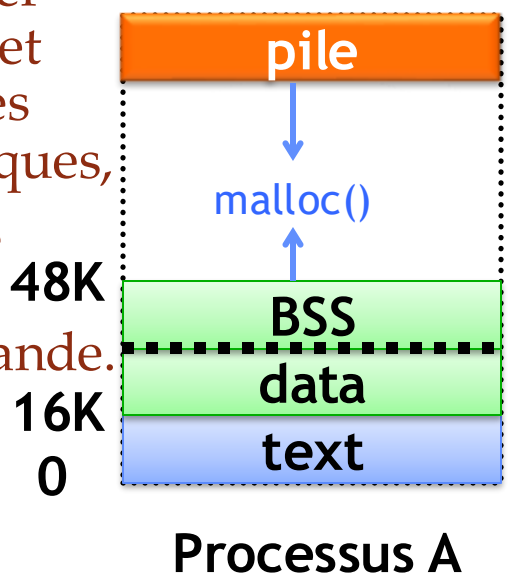
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Modèle mémoire pour un processus

Contraintes : le modèle doit être portable sur des machines hétérogènes qui possèdent des unités de gestion de mémoire (MMU) différentes.

UNIX propose un modèle avec un espace d'adressage à 3 segments :

- Un **segment code (text)** de taille fixe qui sera chargé avec les instructions du programme exécuté en langage machine,
- Un **segment de données (data)** composé d'un premier espace pour les données initialisées par programme et un deuxième espace pour les données non-initialisées (BSS), et un espace libre pour les allocations dynamiques,
- Un **segment pile d'exécution (stack)** qui sera chargé avec les variables d'environnement exportées par le shell ainsi que les arguments de la ligne de commande. Cette pile contient aussi un espace dynamique pour le code des fonctions en cours d'exécution et les registres (machine d'exécution assembleur).



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Mémoire et UNIX

L'appel système **malloc()** permet d'allouer un espace de données pour des variables dynamiques dans le **segment data**. S'il n'y a plus d'espace contigu alors le segment est relogé ailleurs (par suite d'une interruption matérielle).

La **pile** est une interface entre le système et le processus. Ce segment ne peut être modifié que par programme. Elle contient toutes les informations nécessaires à la gestion du processus : registres, adresses des segments, …

Ce segment sert aussi à l'exécution des fonctions du processus.

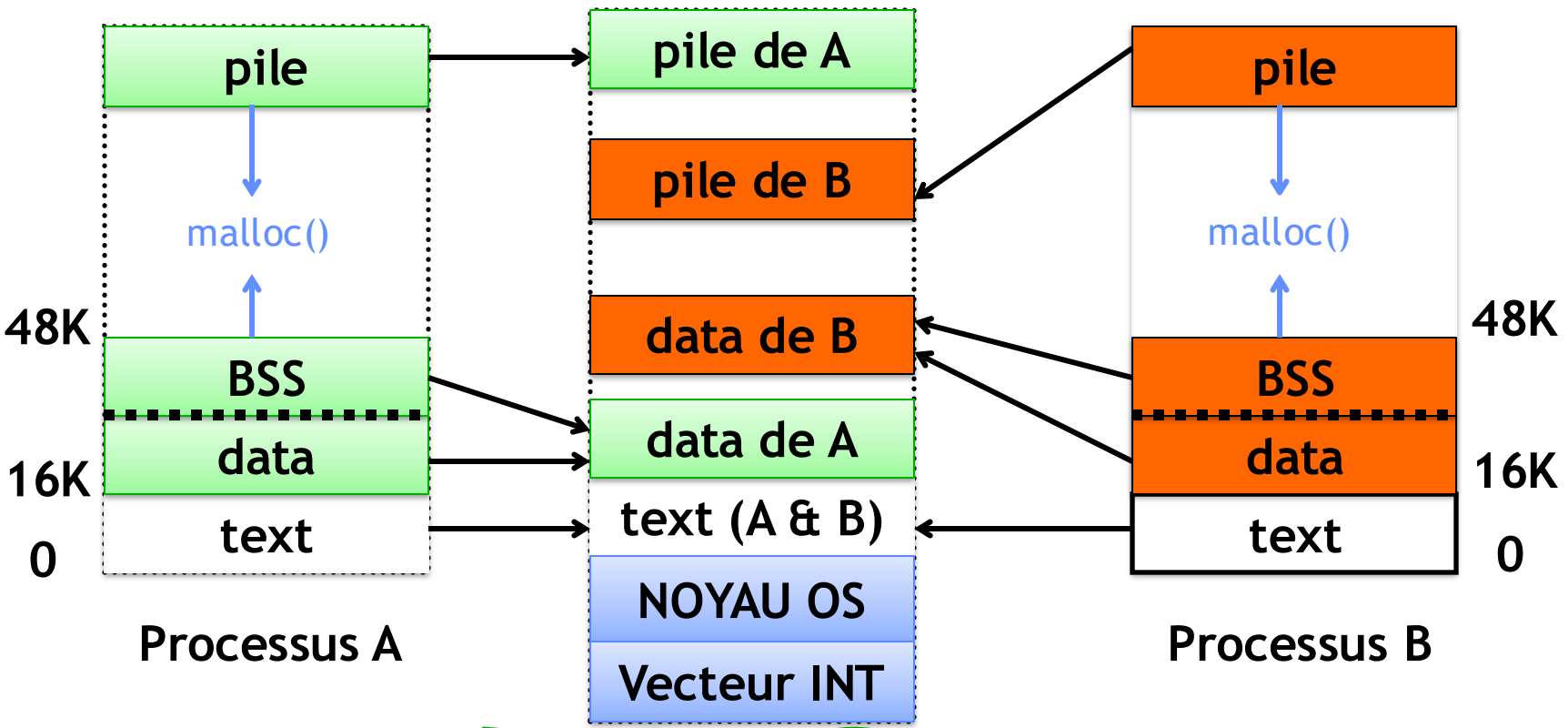
La pile est séparée en deux parties : une pour l'exécution en mode user et l'autre en mode noyau pour les appels système.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

UNIX & réentrance

Le modèle mémoire UNIX supporte la **réentrance** permettant ainsi à plusieurs processus de partager un même **segment mémoire text**.

- Plusieurs demandes d'exécution d'un même programme ne génèrent pas plusieurs chargements : une **seule** copie du programme est effectuée.
- Chaque processus (instance d'un même programme) a sa propre **zone de données** et sa propre **pile d'exécution**



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

ORDONNANCEMENT DE PROCESSUS

Etats d'un processus : élection / réquisition

Préemption

Ordonnancement temps partagé & temps réel

Bibliothèque POSIX

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

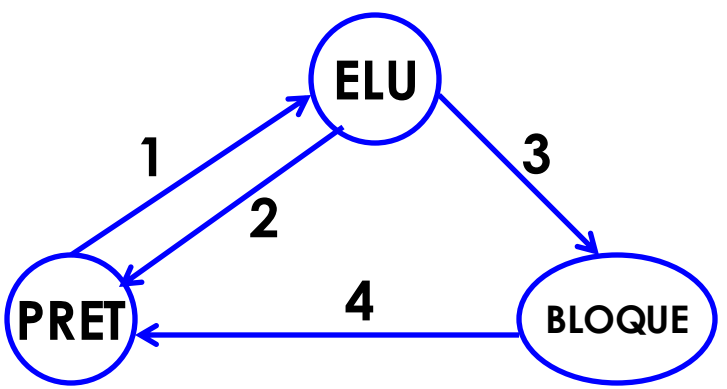
Modèle d'états d'un processus

Un processus possède trois états principaux :

- **ELU** : le processus occupe le CPU
- **PRÊT** : le processus est en attente de l'allocation CPU
- **BLOQUE** : le processus est en attente de données indisponibles

Un processus change d'état suite à la réception d'un signal :

- **Election/Allocation (1)** : le processus est sélectionné par l'ordonnanceur (répartition équitable entre les processus)
- **Réquisition (2)** : le temps CPU alloué est écoulé
- **Blocage (3)** : signal généré quand le processus a besoin de données
- **Réveil (4)** : signal reçu par le processus délivrant les données



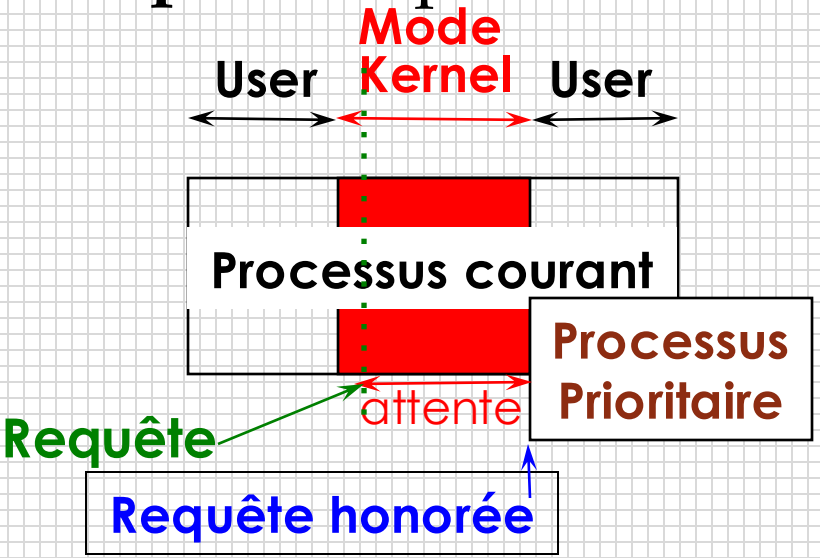
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

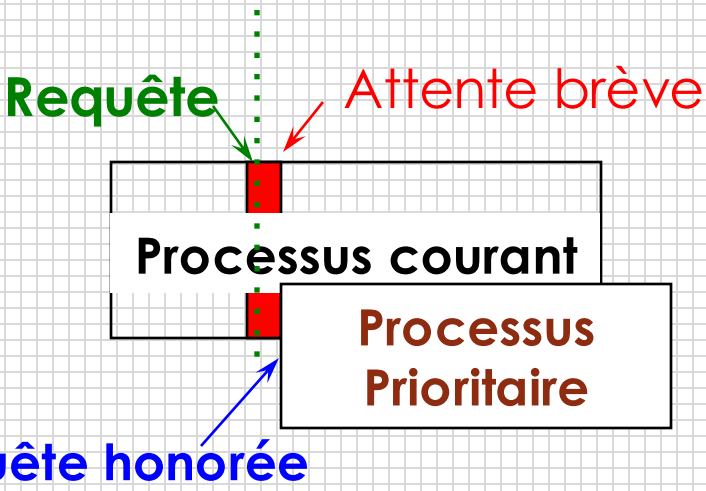
Préemption

Noyau Préemptif : un processus de priorité supérieure peut **interrompre** un processus de priorité inférieure *quelque soit* le mode (**user** ou **kernel**).

- Unix standard & Linux sont **non Préemptifs !**
- Linux peut être **configuré** en mode préemptif.

La **réquisition** est réalisée par l'ordonnanceur pour **préempter** un processus.





© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Ordonnancement des processus

Trois politiques d'ordonnancement cohabitent (POSIX 1003.4) :

- Une politique classique - **temps partagé** (Time Sharing) - : SCHED_NORMAL/SCHED_OTHER ,
- Deux politiques **temps réel** : SCHED_FIFO et SCHED_RR (Round robin = tourniquet).

Ordre de priorité entre les politiques :

- SCHED_FIFO > SCHED_RR > SCHED_NORMAL (voir échelle).

Chaque processus se voit attribuer une politique et une priorité. Cette priorité est **fixe** pour FIFO et RR. En revanche, un processus de type NORMAL a une priorité **dynamique** en fonction de sa consommation UC passée (**équité**).

L'ordonnancement réalisé par le noyau est découpé en cycles. Au début de chaque cycle, on calcule le temps CPU attribué à chaque processus : nombre de ticks d'horloge durant lequel un processus peut être exécuté.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Temps partagé – SCHED_NORMAL

Deux niveaux algorithmiques :

- Niveau **bas** : **élection** d'un processus à exécuter parmi les processus prêts (candidats).
- Niveau haut : va et vient entre le disque et la mémoire pour donner une chance à tous les processus de s'exécuter. Les processus temps réel ne sont jamais swapée sur disque.

On utilise une file d'attente pour regrouper les processus candidats (en mémoire) et ayant le même niveau de priorité :

- Mode **noyau** : priorité **négative** : -20 à -1
- Mode **utilisateur** : priorité **positive** : 0 à 19.

L'ordonnanceur de bas niveau parcourt la table des files d'attente en partant de la priorité la plus haute (la valeur la plus négative), jusque'à trouver une file d'attente non vide.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

SCHED_NORMAL (2)

Un processus garde le CPU pendant un temps max dit quantum. A la fin de celui-ci, il est préempté par l'ordonnanceur.

A chaque pulsation d'horloge, on incrémente le compteur UC qui sera ajouté à la priorité pour affaiblir sa priorité. Toutes les secondes, on recalcule les priorités (selon la période d'ordo) :

- Nouvelle priorité = base (0 ou valeur de nice) + utilisation UC/2

Un processus ayant consommé son quantum est mis en queue dans son niveau de priorité et l'ordonnanceur s'exécute de nouveau : gestion en tourniquet pour les processus de même priorité.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

SCHED_NORMAL (3)

Un processus bloqué par un appel système sera retiré de la file d'attente des processus candidats. **L'appel système** bascule en mode **noyau** (priorité négative) :

- -1 : attente de la fin d'un fils
- -2 : attente de la sortie sur terminal
- -3 : attente d'une entrée sur le terminal
- -4 : attente de disponibilité tampon disque
- -5 : attente E/S sur disque
- ….

La priorité des processus en mode noyau est définie par la nature de l'appel système et son incidence sur l'exploitation du système.

Sur occurrence de l'événement attendu, le processus est remis dans la file d'attente des candidats avec sa priorité avant blocage.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Temps réel – SCHED_FIFO

Premier arrivé, premier servi.

Priorité **fixe**.

Parmi les processus ayant la priorité la plus élevée en choisi le **plus âgé de tous**.

Le processus garde le CPU jusqu'à **terminaison** ou **préemption** par un autre processus **plus prioritaire** (autre que l'ordonnanceur). Dans ce cas le processus reprend sa position dans la file.

L'échelle **POSIX** va de 0 à 63 et celle **LINUX** de 1 à 99.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Temps réel – SCHED_RR

Tourniquet à **quantum** de temps entre processus de même priorité.

Priorité **fixe**.

Le processus ayant la plus haute priorité, est élu.

Le processus est **préempté à la fin du quantum** alloué et réintègre la queue de la file correspondant à sa priorité.

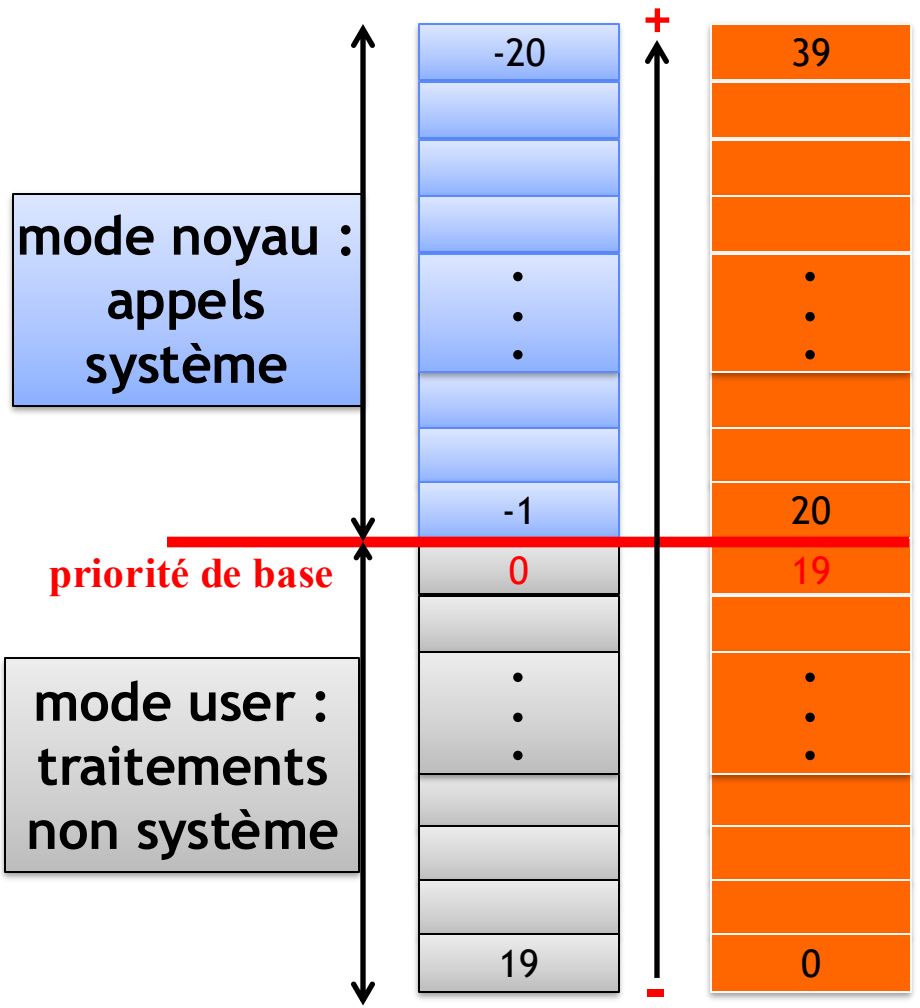
Après préemption d'un processus RR, le processus temps réel ayant la priorité la plus élevée est élu.

La politique RR utilise la **même** échelle que FIFO.

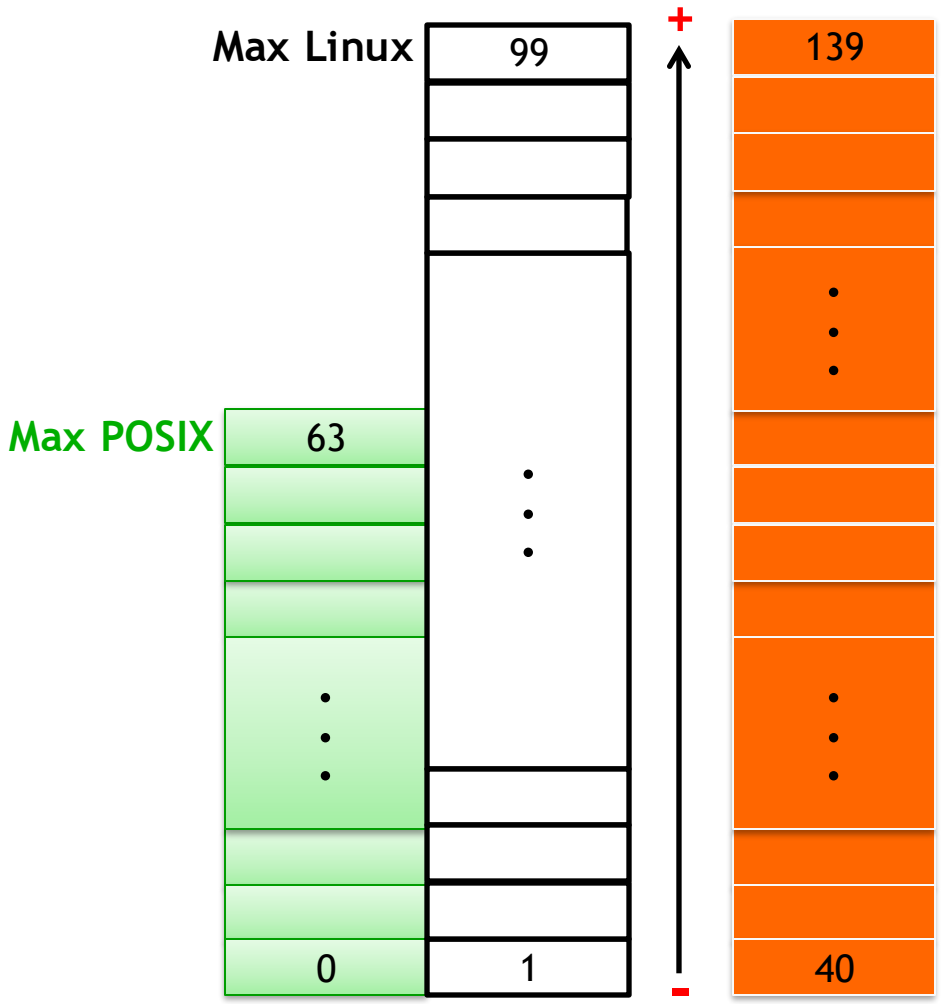
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Echelles POSIX & LINUX pour le TS & le TR

Echelle TS (nice/Linux) :



Echelle TR (POSIX/Linux) :



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Nice Interface

Au niveau du shell :

- À l'exécution du processus :
  - **$ nice [-incrément] command**
    - incrément positif uniquement !
- À l'exécution du processus :
  - **# renice [-n increment] pid**
- pour visualiser ses priorités
  - **$ ps -O user,nice** (A pour tous les processus)
  - **$ ps –O nice,pri**

Par programme :

- En utilisant les routines systèmes standard **nice()** ou plutôt les appels POSIX **setpriority() – getpriority()** pour une portabilité totale.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Realtime Interface

Au niveau du shell :

- pour visualiser ses priorités

```
$ ps -ae0 rtprio
$ ps -ae0 rtprio, pri
```

  - -a et -e est équivalent –A

Par programme :

- Utilisation d'une structure dans sched.h

```
struct sched_param {
    #ifdef POSIX_4D10
        int priority;           /* Scheduling priority. */
    #else
        int sched_priority;     /* Scheduling priority. */
    #endif
};
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Bibliothèque POSIX – Ordonnancement

Appel système	Rôle
sched_getscheduler ()	Retourne la politique d'ordonnancement d'un processus.
sched_getparam ()	Retourne la priorité d'un processus dans sa politique.
sched_get_priority_min ()	Renvoie la valeur de la priorité minimale d'une politique d'ordonnancement.
sched_get_priority_max ()	Renvoie la valeur de la priorité maximale d'une politique d'ordonnancement.
sched_rr_get_interval ()	Quantum pour round-robin

Appel système	Rôle
sched_setscheduler ()	Fixe la politique d'ordonnancement et priorité d'un processus.
sched_setparam ()	Fixe uniquement la politique d'ordonnancement.

Appel système	Rôle
sched_yield ()	Permet de lâcher volontairement le processeur. Très utile pour une politique d'ordonnancement FIFO, beaucoup moins pour une politique RR.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exemple – Librairie ordonnancement de processus

```c
#include <unistd.h>
#include <sched.h>
#include <stdio.h>

// Contrôle des appels système
#define CHECK(sts,msg) if ((sts) == -1) {\
      perror(msg); exit(EXIT_FAILURE); }


void  showInfoSchedFIFO_RR (void) {
// Valeur max de la priorité avec la politique FIFO/RR
      CHECK(int highPriority =
                  sched_get_priority_max(SCHED_FIFO) ,
                  "sched_get_priority_max");
printf("Priorité max pour SCHED_FIFO/RR : %d\n",
                  highPriority);
// Valeur min de la priorité avec la politique FIFO/RR
      CHECK(int lowPriority =
                  sched_get_priority_min(SCHED_FIFO) ,
                  "sched_get_priority_min");
      printf("Priorité min pour SCHED_FIFO/RR :
                  %d\n",       lowPriority);
      return;
}
```

```c
/**
 *   \fn       void  showInfoSchedProcessusCourant(void)
 *   \brief    Affiche les informations d'ordonnancement
       du processus courant
 *   \return   pas de reéultat
 */

void  showInfoSchedCurrentProcess(void) {
      int myPid = 0;       // le processus courant
      struct sched_param param;

      // Valeur courante de la politique
      CHECK(int  myPolicy =
                  sched_getscheduler(myPid),
                  "sched_getscheduler");
      printf("Politique d'ordonnacement du processus :
                  %d\n",       myPolicy);
      // Valeur courante de la priorité
      CHECK( sched_getparam(myPid, &param) ,
                  "sched_getparam");
      printf("Priorité actuelle du processus : %d\n",
                  param.sched_priority);
      return;
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

… Exemple – Librairie ordonnancement de process

```
void  modifySchedCurrentProcess
                (int policy, int priority)
{
      struct sched_param param;

      param.sched_priority = priority;
// Appliquer une nouvelle politique & priorité
      CHECK( sched_setscheduler( 0, policy, &param) ,
                "sched_setscheduler");
}
```

```
#define STATUS system("ps –aO cls,nice,pri,rtprio");

int main (void) {
      showInfoSchedFIFO_RR() ;
      showInfoSchedCurrentProcess() ;
      STATUS;
       modifySchedCurrentProcess(SCHED_FIFO, 99);
      STATUS;
// Mise en attente du processus pour observation
      while (1);
      exit(0);
}
```

Dommage !

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exemple -- Exercices : priorité des processus

Quelle est le politique par défaut et la priorité attribuée à un nouveau processus ?

Quel utilisateur a le droit de changer la politique d'ordonnancement en politique temps réel.

Tester l'exemple précédent, en introduisant une boucle infinie en étant utilisateur privilégié puisque l'appel système **sched_setscheduler ()** ne peut être utilisé par un utilisateur non privilégié.

Utiliser **nice()** pour changer la priorité du processus avec un incrément. Que se passe-t-il, si on utilise un décrément en tant que utilisateur non privilégié mais aussi privilégié ?

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

PROCESSUS

Digramme d'états

Création

Finalisation

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

Implémentation d'un processus ...

A chaque processus, le noyau associe un bloc d'informations système appelé **PCB** (Process Control Block). Le noyau UNIX gère deux structures descriptives, relatives aux processus :

- Une entrée dans la **table des processus** présente en permanence en mémoire :
  - Paramètres d'ordonnancement : Priorité, Temps UC, Temps de sommeil, …
  - Image mémoire : pointeurs sur les différents segments, ou pointeurs sur pages de swap,
  - Signaux : masque des signaux à ignorer ou à délivrer
  - Autres : PID, PPID, UID, GID, Temps avant alarme, événements attendus
- Une **structure** dite **utilisateur** est transférée sur disque en même temps que le processus (swaping) et contient des informations utilisées uniquement en exécution :
  - Les registres de la machine d'exécution : sauvegarde avant appel système,
  - Appel système : paramètres de l'appel, état, résultat produit,
  - Table des descripteurs de fichiers : chaque index (fd) pointe sur l'i-node du fichier associé (cf open())
  - Comptabilité : Temps de calcul consommé, Temps limite, Taille max de la pile, Nombre de pages mémoire max

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

... Implémentation d'un processus

La structure utilisateur est toujours contigüe à la pile (qui se compose d'une pile utilisateur et d'une pile noyau).

La **pile noyau** n'est utilisée que lors d'un appel système qui dispose ainsi de sa propre pile (protection contre des accès illicites).

Le PCB est l'union des informations de la structure utilisateur et des informations de la table des processus.

Création d'un processus par **clonage** :

- Allocation d'une entrée dans la table des processus,
- Duplication de l'entrée père,
- Allocation des segments data et pile (pas de segment text grâce à la réentrance),
- Duplication des segments père dans les segments fils,
- Le fils est prêt pour l'exécution,

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus & Identification

Les processus sont organisés en arbre (même si les données sont enregistrés dans une table).

Chaque processus est identifié par un numéro de processus (**PID**), et possède donc un processus père (**PPID**). Un processus ne connaît le PID d'un fils qu'à sa création.

Les appels **getpid()** et **getppid()** permettent de connaître l'identification attribuée par le système à un processus.

Un **groupe de processus** désigné par un PID ancêtre, est composé de l'ancêtre et tous les processus descendants. Un tel groupe est représenté par **-PID** comme valeur de PID.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus & droits d'accès

Le processus est lancé par défaut avec l'UID/GID de l'utilisateur.

L'**EUID** (**Effective**) prend ainsi par défaut la valeur de l'UID. Il en est de même pour l'**EGID**.

Si le bit SUID est actif sur le programme alors l'**EUID** prend la valeur de l'UID du programme.Il en est de même pour l'**EGID**, si le bit SGID est actif.

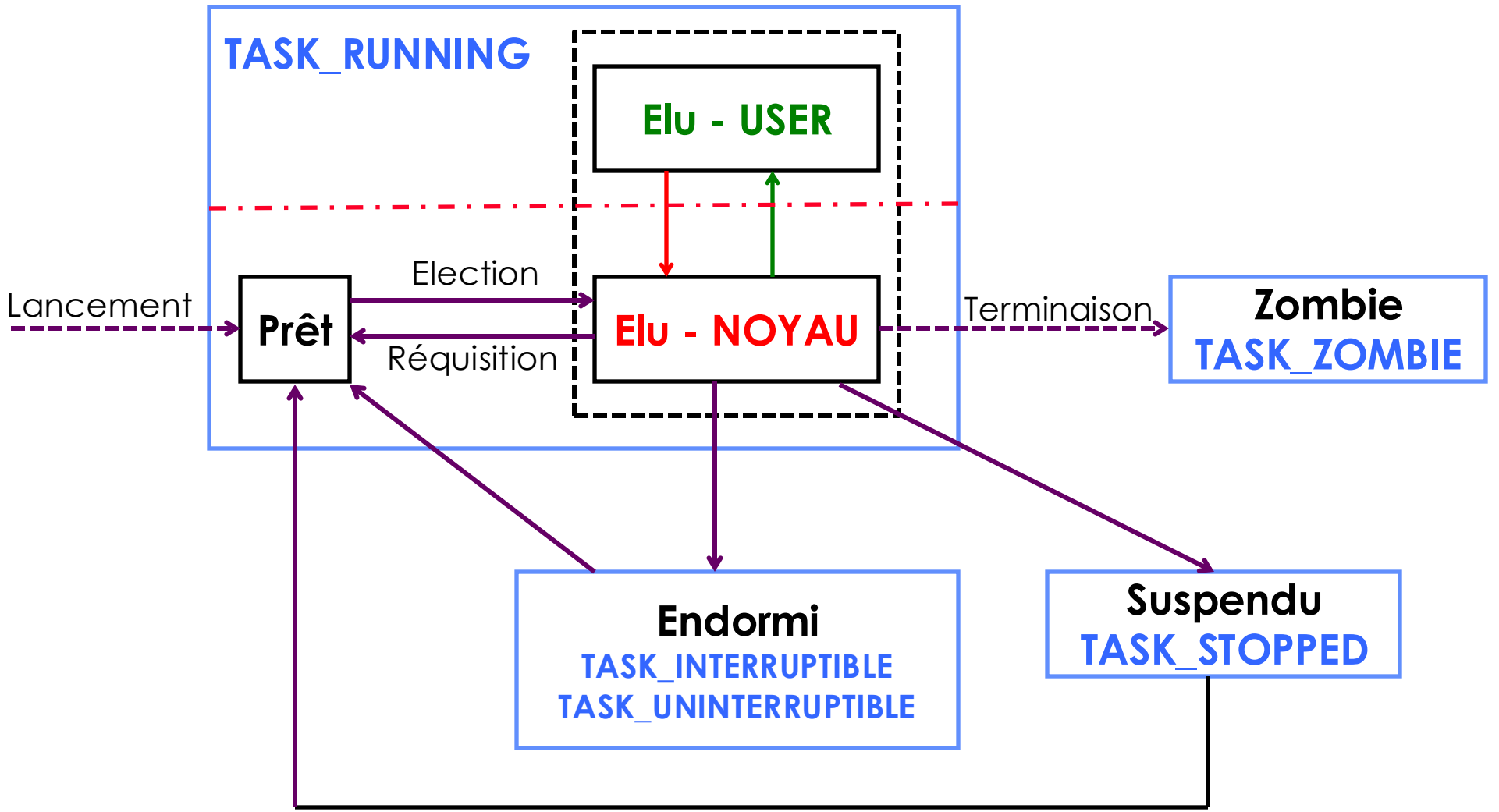
Les appels **getuid()** et **getgid()** permettent de connaître ces valeurs.

L'UID 0 (root) donne accès à un nombre limité et contrôlé d'appels système protégés :

- Accès au bloc 0 d'un disque pour connaître sa taille (interdiction de lire les octets bruts).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Diagramme d'états sous Linux



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Etat des processus – ps …

La commande **ps** : afficher l'état des processus.

- -a : tous les processus de l'utilisateur
- -r : processus "running"
- -l : format long
- -S : temps CPU accumulé
- -u : nom d'utilisateur

Résultats de la commande ps :

- **PID** : processus identificateur
- **TT** : Terminal
- **1° STATUS : état d'exécution.**
  - **R** : **Running**
  - **S** : **Sleep** (moins de 20 secondes)
  - **I** : **Idle** (sleep depuis plus de 20 secondes)
  - **D** : **Down** (bloqué) : processus non interruptible
  - **T** : **Traced** (suivi debug): SIGSTOP active se mode et SIGCONT le désactive
  - **Z** : **Zombie** : terminated and wait for parent
  - **P** : **Page wait**

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

... Etat des processus – ps

Résultats de la commande ps (suite) :

- **2° STATUS : état de la mémoire.**
  - Rien : chargé en mémoire
  - W : processus swapped out
  - > : dépassement mémoire
- **3° STATUS : priorité.**
  - N : priorité réduite
  - < : priorité augmentée
- **TIME** : temps CPU consommé
- **COMMAND** : commande exécutée
- **USER** : propriétaire du processus
- **UID** : user identificateur
- **PPID** : parent processus identificateur
- **PRI** : priorité du processus
- **NI** : priorité affecté par nice
- **SZ** : taille mémoire occupée en Ko

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Création de processus

Un système monotâche lance un processus par **recouvrement** (substitution) du processus courant :

- **execv()** permet ce mode de lancement sous UNIX.
- Exemple : le programme de connexion **login** est substitué par le shell de connexion. A la déconnexion, un autre execv() réactive le login.

Un système multitâche lance un processus par **clonage** du processus courant : **fork()** permet ce mode de lancement.

fork() crée un processus fils à l'image du père (chacun évoluant indépendamment de l'autre par la suite).

La seule différence entre le père et le fils se situe au niveau de la valeur de retour de cette fonction. Ce qui permet de discriminer leur comportement et d'« *émanciper* » le fils :

- 0 dans le fils,
- Numéro du processus créé dans le père (PID du fils).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – introduction au fork()

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <errno.h>

int main (void)
{
   pid_t pidFils;

   pidFils = fork();   /* Demande de création d'un processus */

   switch (pidFils) {
   case -1 :
      perror("Échec de la création d'un processus fils");
      exit (pidFils);

   case 0 :
      /*  On met ici le code spécifique du processus fils */
      exit (0); /* Fin normale du fils  */

   default :
      /* On met ici la suite du code du processus père */
      break;
   }
   return  0;  /* Fin normale du processus père    */
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – Structure de code pour fork()

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <errno.h>

int main(void)
{
   pid_t pidFils;

   pidFils = fork();   /* Demande de création d'un processus */

   if (pidFils == -1) {
      /*  On met ici le code du traitement de l'erreur : cf CHECK */
      perror("Échec de la création d'un processus fils");
      exit (pidFils);
        }
   if (pidFils == 0) {
      /*  On met ici le code spécifique du processus fils */
      exit (0); /* Fin normale du fils  */
        }
   /* On met ici la suite du code du processus père */

   return  0;  /* Fin normale du processus père    */
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – Structure de code pour fork() avec macro CHECK

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <errno.h>

#define CHECK(sts, msg) if (-1 == (sts)) { perror(msg); exit(EXIT_FAILURE); }

int main(void)
{
   pid_t pidFils;
  /* Demande de création d'un processus */
   CHECK(pidFils = fork(), "Échec de la création d'un processus fils");

   if (pidFils == 0) {
      /*  On met ici le code spécifique du processus fils */
      exit (0); /* Fin normale du fils  */
   }
   /* On met ici la suite du code du processus père */
   return  0;  /* Fin normale du processus père    */
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – Structure de code pour fork()

```
#include <unistd.h>
#include <stdlib.h>
#define CHECK(sts,msg) if ((sts) == -1) {perror(msg); exit(-1); }
int main () {
        pid_t pid  = fork();  // clonage
        CHECK(pid, "fork - Problème de création de processus ! ");
        // La valeur du pid dans le père vaut le PID du processus créé (dit fils)
        // Par contre pid vaut 0 dans le processus fils

        if (pid==0) {           // Différenciation entre père et fils
                                // traitement du fils
                exit(0);
                // Il faut prendre l'habitude d'associer systématiquement un exit avec
                // chaque clone, de telle sorte que si on crée des processus en
                // boucle. Les fils ne se mettent pas à leur tour à créer des clones.
        }
        // On continue le traitement spécifique du père ici
        // Un bon père doit attendre la terminaison du fils, qu'il a créé par le biais de
        // wait() ou wait_pid()
        wait();
        exit(0);
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Terminaison d'un processus

Un processus **doit** toujours **finir** avec un exit(), ce qui provoque l'envoi d'un signal (cf. la partie à suivre) au processus père pour lui signifier sa **terminaison**.

Le processus père doit **acquitter** ce signal par un appel système de la famille wait() afin de **libérer** les ressources utilisées par le processus fils.

Sans cet acquittement, le processus devient **Zombie** et **continue à consommer** des ressources.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – Création & finalisation

La création de processus se fait par **clonage** via l'appel système **fork()**. Cet appel invoqué par un processus dit père provoque le clonage de celui-ci en un processus dit fils. Les deux processus continuent leur exécution en **parallèle** à partir du retour de **fork()**.

La fin d'un processus survient suite à l'appel système **exit()** – équivaut le **return** de la fonction **main()** – ou sur **réception** d'un **signal fatal** qu'il ne traite pas.

Le père du processus fils attend sa fin via l'appel **wait()**.

```
pid_t   fork (void);
void    exit (int status);
pid_t   wait(int *status);
pid_t   wait_pid (pid_t  fils, int *status, int options);
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

PROCESSUS MULTITHREADS

Multithreading à l'échelle du processus.
Threads POSIX

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille
IG2I

Parallélisme – Multithreads

Dans les années 80, plusieurs implémentations ont été proposées pour le multitâche léger : commutation au sein du processus alors que d'autres réclament une évolution des primitives classiques pour permettre à plusieurs tâches de partager le même espace mémoire.

Dans les années 90, une uniformisation a donné lieu à une norme **POSIX 1.c** et une bibliothèque **pthreads** (**POSIX threads**).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Threads

Limites du **fork** :

- Pas de partage de variables,
- Création d'un nouveau contexte,
- Commutation de contexte plus lente.

Un thread ressemble beaucoup à un processus fils :

- Pas de création de nouveaux processus,
- Partage de l'espace d'adressage,
- Partage des variables globales et statiques (déclarées localement),
- Partage des descripteurs de fichiers.

Utilisation sous Linux :

```
#include <unistd.h>
#include <pthread.h>
```

- Compilation : **$ gcc myThreads.c –o myThreads -lpthread**

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Mise en place d'un thread ...

On commence par déclarer le traitement du thread sous forme d'une **fonction** dont le **résultat** est de type **void ***, avec **un seul argument** de type **void *** et qui se termine par l'appel **pthread_exit(void **)**

Exemple :

```
void * myThreadFunction (void * arg)
{       // arg est généralement converti par un cast
        // Ici, on décrit le traitement du Thread
        pthread_exit(...);
}

typedef void * (* funcThread_t) (void *);
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

... Mise en place d'un thread

On déclare une **variable** de type pthread_t (type opaque, ne peut être manipulé que par des fonctions pthreads), qui va servir à référencer le thread créé grâce à l'appel :

```
int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
```

Exemple :

```
void main() {
    pthread_t th;
    void * ret;
    pthread_create(&th, NULL, myThreadFunction,
                               valeur_de_arg);
    ....
    pthread_join(th, &ret);
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Thread – Exemple

Ecrire un programme qui crée autant de threads que d'arguments reçus. Chaque argument donnera lieu à un thread chargé de calculer la factorielle de l'argument et de renvoyer son résultat au main (thread principal) .

L'intérêt de ce programme est d'utiliser les différentes primitives de la bibliothèque et de se familiariser avec les paramètres et résultats de celles-ci.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

SIGNALISATION

Interruptions matérielles
Interruptions logicielles
Appels systèmes
Implémentation des signaux
Bibliothèque POSIX

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille
IG2I

INTERRUPTIONS

Interruption & Commutation de contexte

Interruption : IRQ, APIC, E/S avec ou sans interruption

Exceptions : principe, core

Appels système : demande d'un traitement privilégié

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

Commutation de contexte (retour)

3 causes de commutations :

- **Appel système** : demande explicite par la tâche courante,
- **Opération illicite** : on parle de **trappe** ou **exception**. La conséquence directe d'une telle situation est l'arrêt définitif du programme ayant provoqué l'exception :
  - Division par 0,
  - Violation d'accès mémoire,
  - Instruction interdite,
  - …
- Prise en compte d'une **interruption matérielle** par l'OS :
  - Arrêt du programme utilisateur,
  - Exécution de la routine associée à l'interruption,
  - Reprise du programme utilisateur

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Interruption matérielle

Une interruption matérielle est un mécanisme qui demande au système d'**interrompre** le traitement en cours et d'**effectuer** le traitement associé à l'interruption matérielle appelée **routine de traitement de l'interruption**.

La famille Intel 386 admet 15 niveaux d'interruptions (**IRQ0** à **IRQ15**), qui sont gérés par un contrôleur d'interruptions **APIC** (d'antan, deux circuits 8259A cascadés par la broche IRQ2 du 1° circuit).

Un ou plusieurs périphériques sont branchés sur un niveau d'interruption. Si plusieurs, une **scrutation** est nécessaire pour identifier le périphérique concerné.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Noyau – Vecteur d'interruptions

Le système gère une table de type **idt_table** à 256 entrées appelé vecteur d'interruptions :

- Index : numéro de l'interruption,
- Chaque entrée de la table contient l'adresse mémoire du gestionnaire chargé de la prise en compte de l'interruption,
- **0-31** : interruptions non masquables et trappes,
- **32-47** : interruptions matérielles levées par les périphériques,
- **128** : appel système,
- **48-255** (sauf 128) : interruptions programmables.

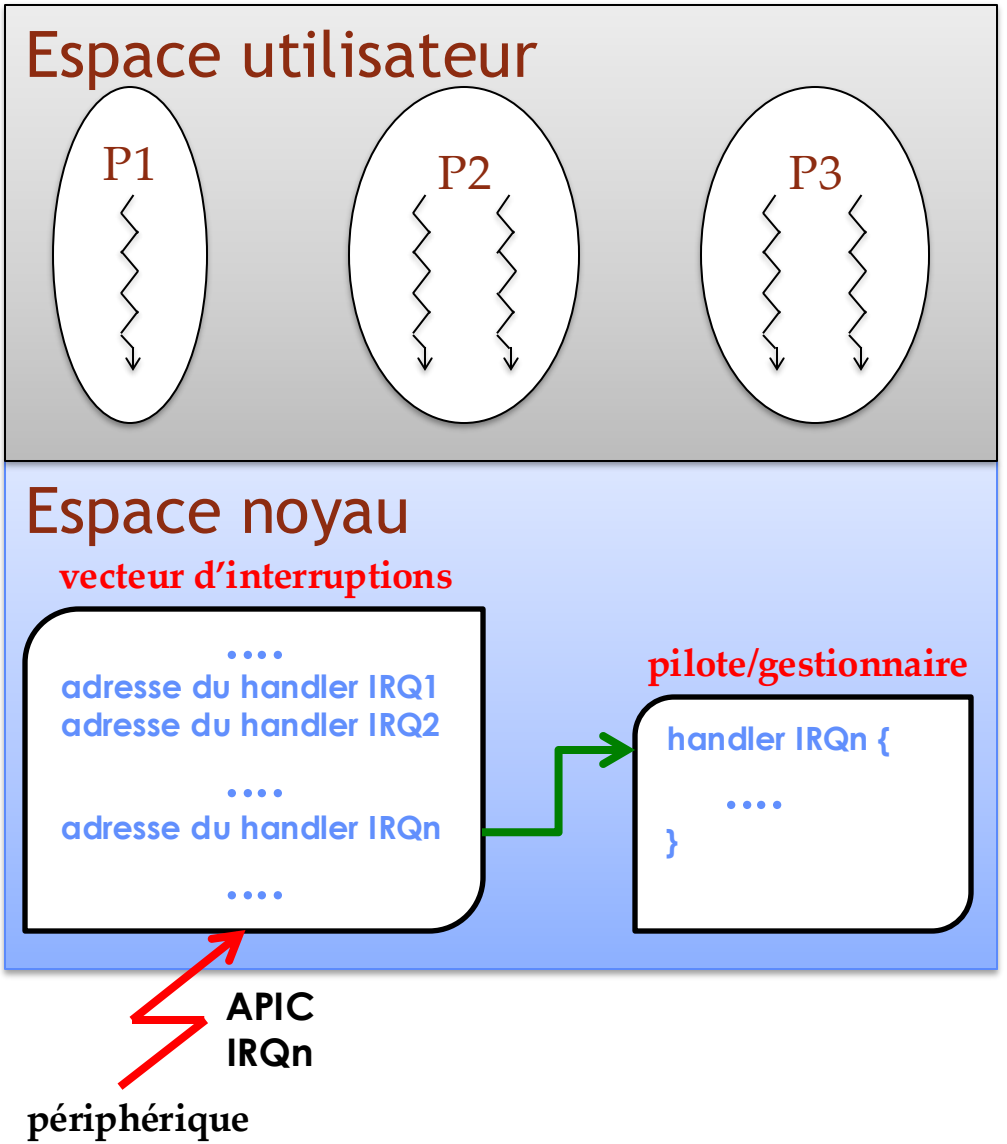
Ce vecteur est chargé en mémoire lors du démarrage du système.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Déclenchement d'une interruption matérielle

A titre d'exemple, prenons le cas d'un bouton poussoir où chaque appui provoque un changement d'état.

Ainsi, l'APIC **identifie** le changement d'état qui correspond soit à un front (0 à 5V) montant ou un front descendant (5V à 0) et **envoie** l'**IRQ** correspondante.



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Prise en compte d'une interruption

L'UC avant d'exécuter une nouvelle instruction vérifie si une **INT** lui a été délivrée. Si présence alors :

- L'UC contrôle l'autorité de l'interruption (vérification de la source),
- L'UC accède à l'entrée dans le vecteur pour récupérer l'adresse mémoire du gestionnaire.
- **Passage en mode noyau** si nécessaire et **commutation** de contexte : sauvegarde du contexte actuel et chargement du contexte gestionnaire.
- **Exécution du gestionnaire de l'interruption**
- Restauration du contexte précédent

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Interruption matérielle – Protocole de traitement

Protocole de traitement :

- Émission par le contrôleur d'un signal sur la broche **INT**,
- Le processeur acquitte sur la broche **INTA**,
- Le contrôleur place le **numéro de l'interruption** sur le bus de données,
- L'UC utilise ce numéro pour **indexer** le vecteur (offset de 32),
- **Exécution du gestionnaire** avec 3 situations possibles :
  - Actions **critiques**, immédiates, interruptions masquées,
  - Actions **non critiques**, exécution rapide, interruptions démasquées,
  - Actions qui peuvent être **différées**, exécution longue, non critiques pour le fonctionnement du noyau,
- Restitution du contexte d'appel :
  - Si le niveau d'imbrication est à 1 alors finalisation des actions différées et retour au mode user.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Interruption logicielle – Exceptions ou trappes

Quand l'UC rencontre une situation anormale, une **exception** est levée (une trappe est ouverte) (on parle aussi d'**interruption logicielle**) :

- Division par 0, Violation d'accès mémoire, Instruction interdite, …

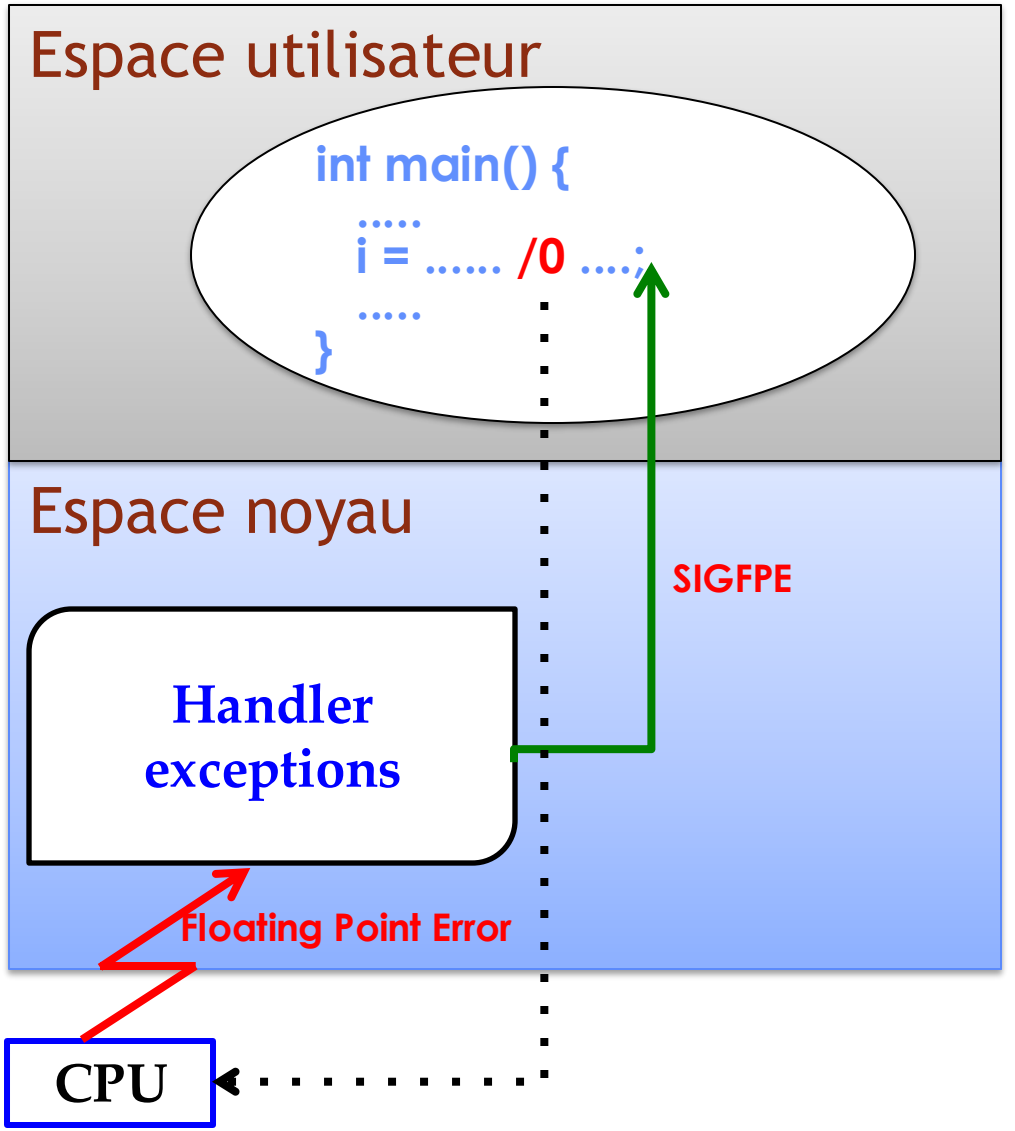
Le traitement d'une trappe est assez similaire à celui des interruptions matérielles. La différence réside dans le fait que le traitement est **synchrone** avec l'exception.

Sur occurrence d'une trappe, un **signal** est envoyé au processus **fautif** qui exécute alors un traitement par défaut (généralement, la fin du processus) ou programmé.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Noyau – Traitement d'une exception

A titre d'exemple, si un processus exécute une instruction qui fait une **division par zéro**, alors le CPU provoque une **interruption interne** que le **Handler d'exceptions** transforme en un **signal** envoyé au processus concerné.



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Appel système

Un **appel système** est invoqué soit par un programme soit par le biais d'une commande.

Protocole de traitement d'un appel système :

- **Copie des paramètres** (maximum 5) dans les registres EBX, ECX, EDX, ESI et EDI,
- **Chargement** du registre EAX avec **le numéro de la routine**, qui sert d'index pour le gestionnaire des appels dans une table d'appels système (256 entrées) chargée avec les adresses des routines,
- **Interruption logicielle** (numéro 128),
- **Sauvegarde** du contexte sur la pile noyau,
- **Passage** en mode noyau,
- **Exécution de la routine** invoquée avec résultat dans EAX,
- **Restauration** du contexte

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

PROGRAMMATION DES SIGNAUX

Notion de signal : classiques & TR

Masque de signaux à délivrer pou un processus

Traitement/déroutement d'un signal

Cas des signaux TR

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

Notion de Signal

Le **noyau** sur occurrence d'un **événement**, génère un **signal** à destination d'un **processus** ou d'un groupe de processus.

L'événement peut avoir :

- une origine **matérielle** (interruption matérielle) généralement traité par un gestionnaire (en mode noyau) qui génère un signal.
- une origine **logicielle** (interruption logicielle) : exception ou envoi d'un signal par un autre processus (vérification des droits et régénération par le noyau).

Un signal est un **message** envoyé par le noyau à un processus :

- après validation éventuelle des droits, si la source n'est pas le noyau.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Implémentation d'un signal sous UNIX

UNIX propose une liste de signaux **nommés** :

- **signaux classiques :**
  - Les **signaux classiques** sont **numérotés** de 1 à 31.
  - La numérotation **dépend** de l'implémentation.
  - L'utilisation des **noms symboliques** garantit la portabilité du code.
  - Les signaux portent un nom symbolique préfixé par SIG :
    - SIGINT, SIGKILL, SIGSTOP, SIGFPE, SIGSEGV, SIGCHLD,
- **signaux temps-réel** :
  - Les **signaux temps réel** sont **numérotés** de 32 à 63,
  - SIGRTMIN, SIGRTMIN+1, ..., SIGRTMIN+14,
  - SIGRTMAX-15, ..., SIGRTMAX-1, SIGRTMAX,

L'utilisation des noms symboliques est fortement recommandée.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

signal.h …

```
/* * valid signal values: all undefined values are reserved for future use
•note: POSIX requires a value of 0 to be used as the null signal in kill() */

#define SIGHUP    1     /* hangup, generated when terminal disconnects */
#define SIGINT    2     /* interrupt, generated from terminal special char */
#define SIGQUIT   3     /* (*) quit, generated from terminal special char */
#define SIGILL    4     /* (*) illegal instruction (not reset when caught)*/
#define SIGTRAP   5     /* (*) trace trap (not reset when caught) */
#define SIGABRT   6     /* (*) abort process */
#define SIGEMT    7     /* (*) EMT instruction */
#define SIGFPE    8     /* (*) floating point exception */
#define SIGKILL   9     /* kill (cannot be caught or ignored) */
#define SIGBUS    10    /* (*) bus error (specification exception) */
#define SIGSEGV   11    /* (*) segmentation violation */
#define SIGSYS    12    /* (*) bad argument to system call */
#define SIGPIPE   13    /* write on a pipe with no one to read it */
#define SIGALRM   14    /* alarm clock timeout */
#define SIGTERM   15    /* software termination signal */
#define SIGURG    16    /* (+) urgent contition on I/O channel */
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

… signal.h

```
#define SIGSTOP    17      /* (@) stop (cannot be caught or ignored) */
#define SIGTSTP    18      /* (@) interactive stop */
#define SIGCONT    19      /* (!) continue if stopped */
#define SIGCHLD    20      /* (+) sent to parent on child stop or exit */
#define SIGTTIN    21      /* (@) background read attempted from control terminal*/
#define SIGTTOU    22      /* (@) background write attempted to control terminal */
#define SIGIO      23      /* (+) I/O possible, or completed */
#define SIGXCPU    24      /* cpu time limit exceeded (see setrlimit()) */
#define SIGXFSZ    25      /* file size limit exceeded (see setrlimit()) */
#define SIGVTALRM 26       /* virtual time alarm (see setitimer) */
#define SIGPROF    27      /* profiling time alarm (see setitimer) */
#define SIGWINCH   28      /* (+) window size changed */
#define SIGINFO    29      /* (+) information request */
#define SIGUSR1    30      /* user defined signal 1 */
#define SIGUSR2    31      /* user defined signal 2 */
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Signaux classiques …

Si le processus n'est pas élu alors le signal est **mis en attente**. Plusieurs occurrences d'un même signal sont réduites à **une seule occurrence**.

Si le processus est élu alors le signal est **délivré** de suite.

La délivrance d'un signal à un processus provoque une **interruption logicielle** par une réaction immédiate. Ce comportement **synchrone** (à la livraison) se concrétise par l'exécution du traitement associé au signal délivré : on parle de déroutement de processus.

- Un processus, sur réception du signal, exécute une routine désignée par un « **signal handler** ».
- Chaque processus dispose d'un vecteur de signaux qui associe à chaque signal un traitement donné. Ce vecteur est créé et initialisé avec les traitements par défaut (cf. man 7 signal).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Signaux classiques

Un processus a la possibilité de **bloquer** un signal :

- c-à-d ne pas être **réceptif** à un signal en les bloquant.
- Certains signaux ne peuvent être bloqués.
- Chaque processus dispose d'un **masque** de signaux (table) qui caractérise la réceptivité de chaque signal.

Attention : Ne pas confondre interruptions et signaux.

Classification des signaux :

- Système ;
- Terminal et I/O ;
- Exception logicielle ;
- Alarmes ;

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – Traitement d'un signal

Pour traiter un signal, un processus peut :

- **bloquer** le signal (certains signaux ne peuvent être bloqués),
- **programmer** le signal : le traitement principal est interrompu, et une procédure de traitement du signal prend le relais avant la reprise de celui-ci (traitement principal). – cf plus loin –
  Le traitement d'un signal peut être à son tour interrompu par une autre signal.
- ou réaliser le **traitement par défaut** (en général, tuer le processus).

Traitements (par défaut) proposés par le système	
TERM	Terminaison du processus.
CORE	Terminaison du processus avec une image de la mémoire du processus en vue du déverminage.
STOP	Arrêt du processus pour une reprise par **SIGCONT**.
CONT	Reprise d'un processus arrêté par **SIGSTOP** ou **SIGSTP**.
IGN	Ignorer le signal par le processus (aucun traitement).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Signaux – Système

Les signaux système **SIGKILL/ SIGSTOP/SIGCONT ne peuvent-être masqués, ni ignorés, ni reprogrammés :** traitement prédéfini.

Certains signaux système **ne peuvent pas être masqués mais reprogrammés : SIGCHLD, ….**

Signaux système			
SIGABRT	6	CORE	Signal provoqué par l'appel **abort()**.
SIGKILL	9	TERM	Terminaison du processus : signal KILL.
SIGSYS	12	CORE	Erreur dans le paramètre d'un appel système.
SIGTERM	15	TERM	Terminaison du processus : signal logiciel.
SIGSTOP	17	STOP	Arrêt du processus pour une reprise par **SIGCONT**.
SIGCONT	19	CONT	Reprise d'un processus arrêté par **SIGSTOP/SIGSTP.**
SIGCHLD	20	IGN	Terminaison d'un processus. Ce signal est traité uniquement par l'appel système **wait()**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Signaux – Terminal & I/O

Signaux Terminal & I/O			
SIGHUP	1	TERM	Déconnexion du terminal : CTRL-D.
SIGINT	2	TERM	Un break/cancel par clavier est demandé : CTRL-C.
SIGQUIT	3	CORE	Demande clavier de terminaison.
SIGPIPE	13	TERM	Canal "pipe" brisé.
SIGURG	16	IGN	Données urgentes présentes sur un canal d'E/S.
SIGSTP	18	STOP	Arrêt du processus demandé par une séquence clavier : CTRL-Z.
SIGTTIN	21	IGN	Données terminal en entrée (background).
SIGTTOUT	22	IGN	Données terminal en sortie (background).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Signaux – Exceptions, alarmes, ...

Signaux d'exception			
SIGILL	4	TERM	Instruction illégale.
SIGFPE	8	TERM	Erreur de calcul mathématique.
SIGSEGV	11	TERM	Violation segment mémoire.
SIGXCPU	24	TERM	Dépassement consommation CPU (cf. **setrlimit()**).
SIGXFSZ	25	TERM	Dépassement taille fichier (cf. **setrlimit()**).

Signaux d'alarme			
SIGALRM	14	TERM	Fin d'un timer armé par **alarm()**.
SIGVTALRM	26	TERM	Fin d'un timer virtuel armé par **setitimer()**.
SIGPROF	27	TERM	Fin d'un timer de profiling armé par **setitimer()**.

Signaux libres (programmables)			
SIGUSR1	30	TERM	Signal sans signification particulière.
SIGUSR2	31	TERM	Signal sans signification particulière.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Signaux – Appels système principaux

Envoi de signal à un processus : **kill()**.

Réceptivité d'un processus aux signaux :

- Notion de masque de signaux.
- Le processus hérite du masque de son père.
- Pour modifier le masque courant, il faut préparer un masque de signaux à bloquer ou à débloquer par **sigprocmask()**. Ainsi, on redéfinit la réceptivité du processus vis-à-vis de chaque signal :
  - sur occurrence d'un signal, le processus sera soit **réceptif** (débloqué) ou **non** (bloqué).

Traitement d'un signal :

- Le processus hérite du traitement de signal du père.
- Pour modifier le traitement actuel, il faut préparer un **signal handler** qui définit le traitement à effectuer sur réception du signal et l'appliquer par **sigaction()**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Envoi de signal – Interactive kill

$ kill -l

- liste les signaux gérés par le système

$ kill [-signal_name | -signal_number] pid …

- **$ kill -9 6996** équivalent à **$kill -KILL 6996**
- **$ kill -KILL 0**
  - Cette commande envoie SIGKILL à tous les processus appartenant au « shell process group »
- **$ kill -USR1 6996**

Appelation trompeuse : **kill** délivre un **signal** en général et pas seulement pour tuer un processus (un signal parmi d'autres).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Envoi de signal – kill()

Appel système	Rôle
#include <sys/types.h> #include <signal.h>	
int kill (pid_t pid, int sig)	Envoi du signal de numéro **sig** au processus ayant comme PID **pid**. Si **pid=0** alors le signal est envoyé au processus émetteur. Si **pid<=0** alors le signal est envoyé à un groupe de processus dont l'ancêtre porte **-pid** (**| pid |**).
int raise (int sig)	Envoi du signal de numéro **sig** au processus lui-même.

Parricide :

```
if ( kill(getppid(), SIGTERM) == -1 )
        perror("Error in kill");
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – Préparation d'un masque de signaux

Tout processus dispose d'un **masque par défaut** hérité du processus père.

Afin de modifier ce masque, il faut **préparer** un autre masque avant de l'appliquer par **sigpromask()**.

Appel système	Rôle

`#include <sys/types.h>`
`#include <signal.h>`
`// Définit le type opaque sigset_t pour représenter un masque (table) de signaux`	

`int sigemptyset (sigset_t *set)`	Vide le masque **set** fourni en paramètre.
`int sigefillset (sigset_t *set)`	Rempit le masque **set** fourni en paramètre.
`int sigaddset (sigset_t *set, int signo)`	Ajoute le signal de numéro **signo** du masque **set**.
`int sigdelset (sigset_t *set, int signo)`	Supprime le signal de numéro **signo** du masque **set**.
`int sigismember (sigset_t *set, int signo)`	Vérifie la présence du signal de numéro **signo** dans le masque **set**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – Application d'un masque de signaux

Pour rendre opérationnel un nouveau masque, on utilise l'appel système **sigprocmask().**

```
#include <signal.h>
int sigprocmask(int how, const sigset_t *set, sigset_t *oset);
```

How :

- **SIG_BLOCK** : ajoute les signaux du masque au masque courant
- **SIG_UNBLOCK** : retire les signaux du masque courant
- **SIG_SETMASK** : affecte et remplace le masque courant

set : valeur du masque (peut être NULL)

oset : ancienne valeur (peut être NULL)

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exemple – Mise en œuvre d'un masque de signaux

```
#include <unistd.h>
#include <signal.h>
#define CHECK(sts,msg) if ((sts) == -1) { perror(msg); exit(-1); }
int main () {
        sigset_t intmask;
        int sts;
        // Initialise et affecte le masque
        sts = sigemptyset(&intmask);
        CHECK(sts, "appel de sigemptyset");
        sts = sigaddset(&intmask, SIGINT);
        CHECK(sts, "appel de sigaddset");
        // On interdit le control-C
        sts = sigprocmask(SIG_BLOCK, &intmask, NULL);
        CHECK(sts, "appel de sigprocmask");
        // On attend pour tester
        alarm(60);
        pause();
        exit(0);
}
```

L'appel alarm() arme une alarme et continue le traitement. Sur fin de temporisation SIGALRM est reçu par le processus.
L'appel pause() se débloque sur réception d'un signal qcq

Le programme dure 60 secondes et quitte; pendant ce temps le control-C est inhibé.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – traitement d'un signal

La mise en place du traitement d'un signal se fait en 2 étapes :

- Préparation d'une structure **sigaction** en renseignant son attribut **sa_handler** :
  - Nom de la **routine utilisateur** pour le traitement du signal. Cette routine ne prend qu'un **seul** argument de type int et il prend la **valeur du signal** ayant provoqué l'interruption.
  - **SIG_DFL** : action par défaut
  - **SIG_IGN** : nop
  - La commande **man 7 signal** permet de connaître le traitement par défaut de chaque signal
- Association de la structure au signal par l'appel système **sigaction()**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus – Traitement d'un signal

struct sigaction		
void	(*sa_handler) ()	Nom de la routine à lancer sur occurrence du signal. La routine respecte le prototype void handler (int signo). Autres valeurs possibles : SIG_DFL/SIG_IGN.
sigset_t	sa_mask	Masque de signaux pendant le traitement du signal. Quand le processus traite le signal, il peut être interrompu par un autre signal.
int	sa_flags	Paramétrage du comportement pendant le traitement.

Appel système	Rôle
#include <sys/types.h> #include <signal.h> // Déclare la structure sigaction pour désigner le traitement d'un signal	
int sigaction (int signo, const struct sigaction *new_act, struct sigaction *old_act);)	Associe au signal signo le traitement définit par new_act et sauvegarde le traitement actuel dans old_act. Ce dernier peut valoir NULL si l'on ne souhaite pas sauvegarder.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exemple – traitement de signal

```
#include <signal.h>
#define CHECK(sts,msg) if ((sts) == -1) {perror(msg); exit(-1); }

struct sigaction newact;
void monSIGUSR1(int numero_signal);

newact.sa_handler = monSIGUSR1;
sigemptyset(&newact.sa_mask);
newact.sa_flags = 0;
CHECK(sigaction(SIGUSR1, &newact, NULL),
                "problème avec sigaction");
```

La routine de traitement d'un signal (ici monSIGUSR1) n'accepte qu'un seul paramètre qui est affecté avec le numéro du signal déclencheur.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exemple – déroutement d'un signal

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>

#define CHECK(sts,msg) \
        if ((sts) == -1) {perror(msg); exit(-1); }

int main()
{
    struct sigaction newact;
    int sts;
    void deroute();

    newact.sa_handler = deroute;
    CHECK(sigemptyset(&newact.sa_mask),
        "problème sigemptyset");
    newact.sa_flags = 0;

    // on installe le handler deroute
    CHECK(sigaction(SIGUSR1, &newact, NULL),
        "problème sigaction");

    // on attend les signaux
    for( ; ; ) {
        pause();
        printf("On est passe par ici :
        pause se réveille sur un signal.\n");
    }

    exit(0);
}

void deroute (int signal_number)
{
    switch (signal_number) {
    case SIGUSR1 : printf("Signal SIGUSR1
        reçu.\n"); break;
    case …..
    }
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Appels annexes

alarm() : déclenche **SIGALRM** après tempo

pause() : attend l'occurrence d'un signal

wait() : attend les enfants

waitpid() : attend un processus fils donné

sleep() et **nanosleep()** : suspend le traitement pendant la durée (paramètre) et se déclenche sur occurrence d'un signal de fin de sommeil.

De toute manière, il faut toujours vous référer au **man** de **votre** système.



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

SYNCHRONISATION & INTERBLOCAGE

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille
IG2I

Conception/modélisation – RdP & StateCharts

Conception & modélisation des systèmes parallèles (le cas de la plupart des systèmes temps réel).

Un autre outil de modélisation est les **StateCharts** (**Yakindu** est une implémentation sous **Eclipse**). Un StateCharts est un automate à états finis, enrichi par la hiérarchisation des états et l'orthogonalité des automates avec une diffusion synchrone des évènements.

Système TR :

- Ensemble de tâches parallèles.
- Analyse formelle pour la validation comportementale (unitaires),
- Validation temporelle par mesures & simulation.
- L'intégration se base sur la **synchronisation** des tâches (& ordo).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Réseaux de Petri

G = (P, T, A, M)

P est l'ensemble des sommets **place** alors que T est l'ensemble des **transitions**.

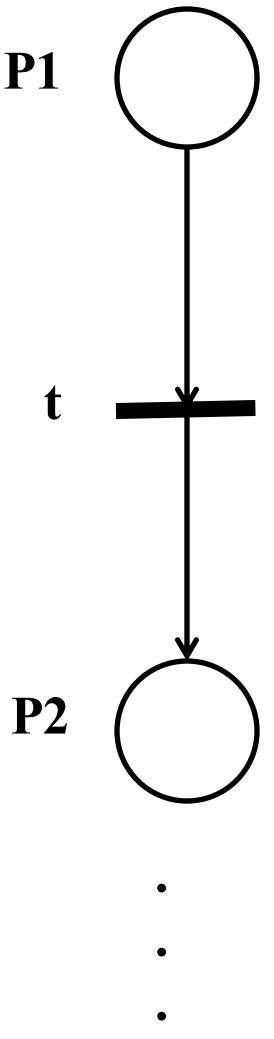
Dans un RdP, le passage d'une place à une autre se fait en franchissant une transition (ensemble A). Généralement, une transition désigne les conditions de passage. Les places modélisent un état ou une action.

Un RdP est munit d'un marquage initial (M) :

- Une marque est un jeton qui modélise la dynamique du système. Un arc peut être pondéré par le nombre de marques consommées ou le nombre de marques produites.

Modélisation du comportement /fonctionnement d'un processus.

Validation formelle par analyse de propriétés (outils mathématiques).



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

RdP – mécanismes de communication/synchronisation

Plusieurs schémas classiques de communication interprocessus ont été élaborés en RdP pour modéliser le partage entre processus :

- **Synchronisation** (cf. plus haut) :
  - Avec ou sans acquittement ,
  - rendez-vous.
- Accès en **exclusion mutuelle** à une ressource.
- **Producteurs/Consommateurs** (matière) :
  - Rédacteurs/lecteurs (information),
  - La différence entre les 2 modèles se situe au niveau du nombre de consommateurs pour une unité produite
- Requêtes / Réponses.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

SYNCHRONISATION

Synchronisation & Ressource

Implémentation d'une exclusion mutuelle : Dekker, TSL,

Généralisation – les sémaphores de Dijkstra

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

Synchronisation

Hypothèse :

- Les processus sont complétement asynchrones.

Objectifs : franchissement simultanée ou non d'une barrière

- Bloquer un processus ou soi-même,
- Activer un processus avec ou sans mémorisation.

Réalisation : Deux techniques

- Synchronisation directe : on nomme explicitement le processus,
- Synchronisation indirecte : on actionne un mécanisme qui agit sur d'autres processus.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

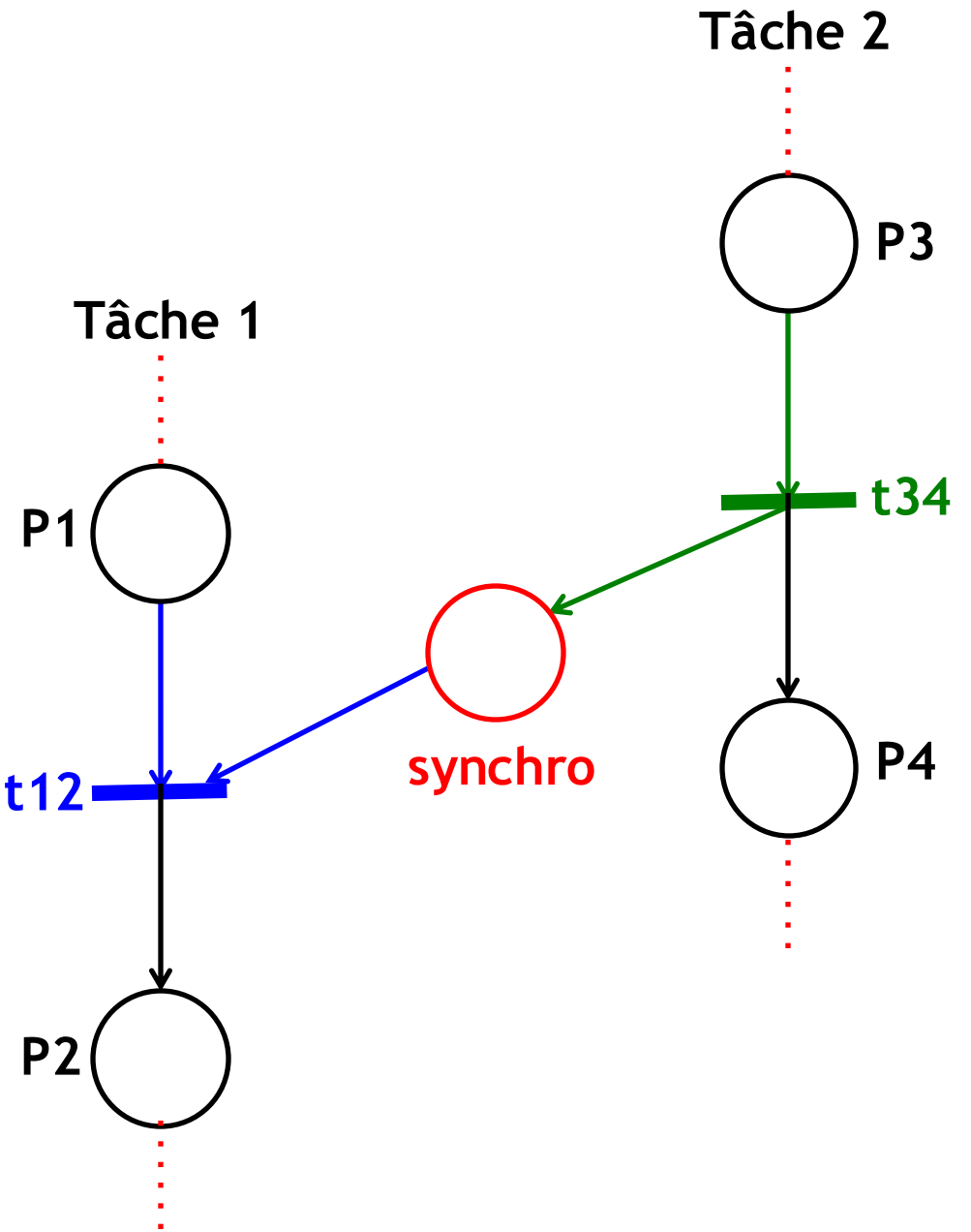
Synchronisation – modèle

Une **ressource** de **synchronisation** permet à une tâche de se synchroniser sur **un état atteint** par une autre tâche.

Ainsi, la tâche 1 ne peut se trouver en P2 que si la tâche 2 a atteint P4. Ceci implique que la transition t34 a été franchie et qu'une marque de synchro a été produite.

Si la tâche 1 était en P1 (attente) alors le passage à P2 est immédiat et la synchro sera consommée.

Sinon, la synchro sera conservée jusqu'à ce que la tâche 1 soit en P1.



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Synchronisation – Exemple 1

t0 (État initial) : P1 est en **attente** de synchro et P3 vient d'être marquée :

P1 & P3 marquées.

t1 (tir de t34) :

P3 perd sa marque,

Synchro est marquée,

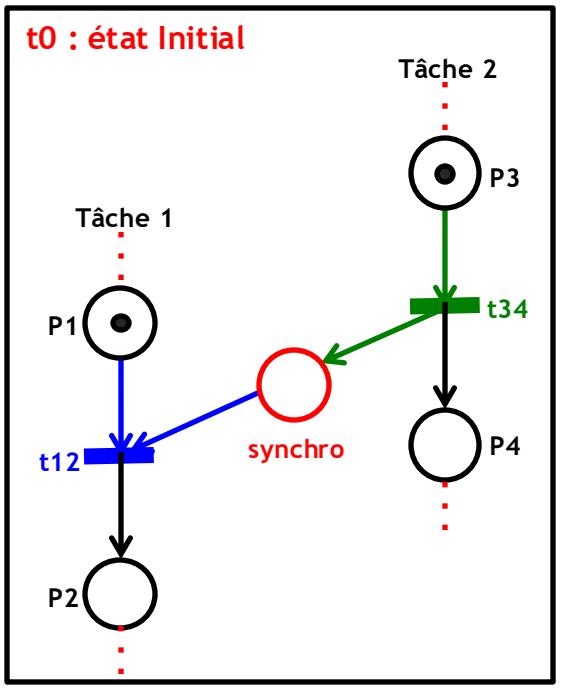
P4 est marquée.

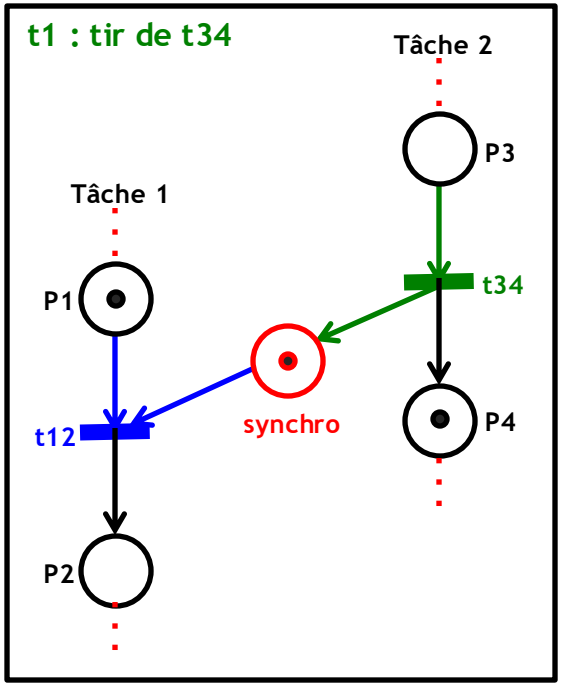
t2 (tir de t12) : P1 était marquée alors il y a tir **immédiat** :

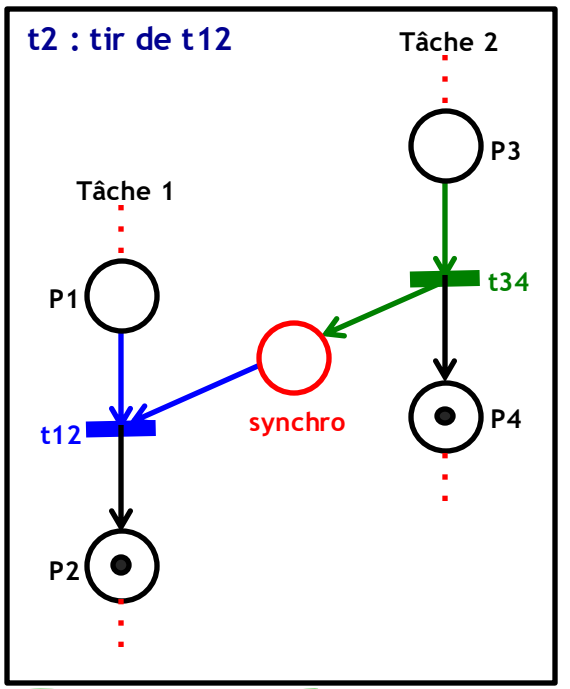
P1 perd sa marque,

Synchro perd sa marque,

P2 est marquée.







© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Synchronisation – Exemple 2

t0 (État initial) : P3 vient d'être marquée :

P3 marquée.

t1 (tir de t34) :

P3 perd sa marque,

Synchro est marquée,

P4 est marquée.

t2 (marquage de P1) : la tâche 1 produit une marque dans P1 :

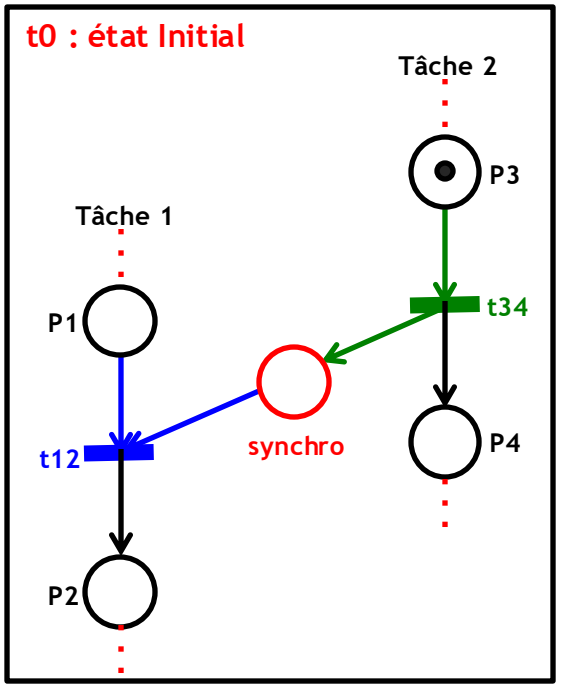
P1 marquée.

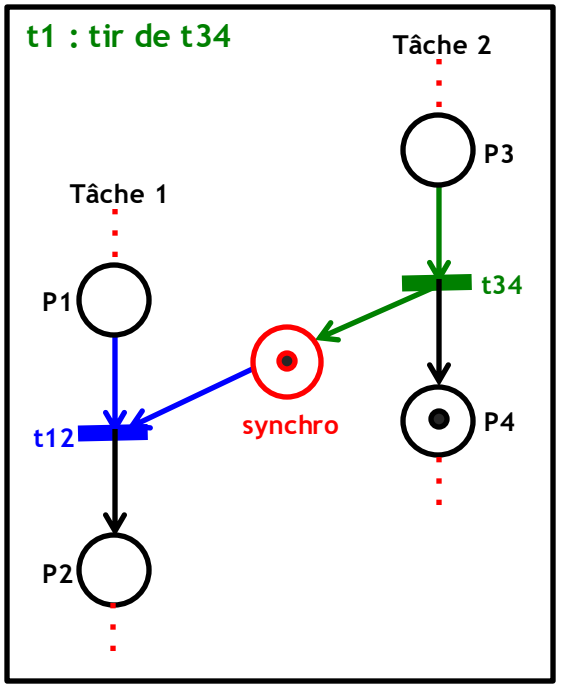
t3 (tir de t12) : dès que P1 est marquée, il y a tir de t12 :

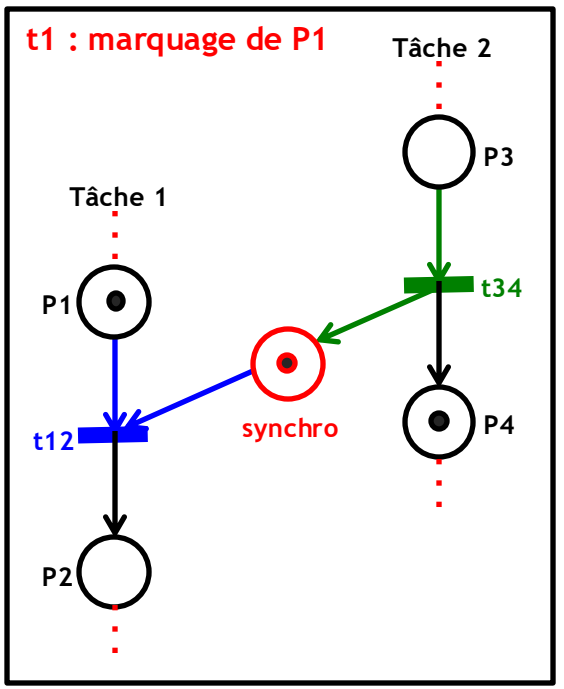
P1 perd sa marque,

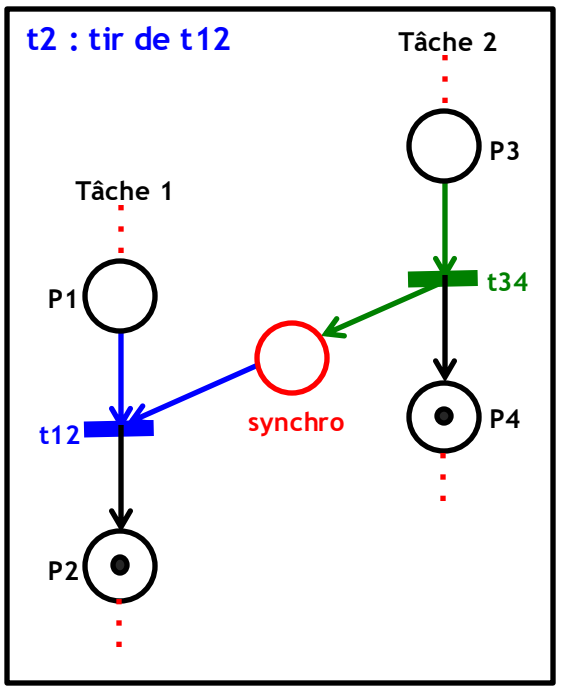
Synchro perd sa marque,

P2 est marquée.









© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

RdP – Notion de Ressource

La notion de ressource recouvre toute entité nécessaire à l'exécution d'un processus. Elle peut être :

- **Matérielle** : un processeur, une imprimante, un écran, …
- **Logicielle** : une variable, une zone mémoire,

Le noyau système garantit :

- La **protection des données utilisateurs** : tous les segments mémoire d'un processus ne sont accessibles que par celui-ci.
- Les **appels système** donnent un **accès exclusif** aux **ressources système** qu'elles soient matérielles ou logicielles.

Dès l'instant où une ressource est **partagée** par plusieurs processus alors on se retrouve en situation de **concurrence** :

- Nécessité de mécanismes de protection pour permettre le partage entre processus utilisateurs (droits d'accès).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Ressource & Synchronisation

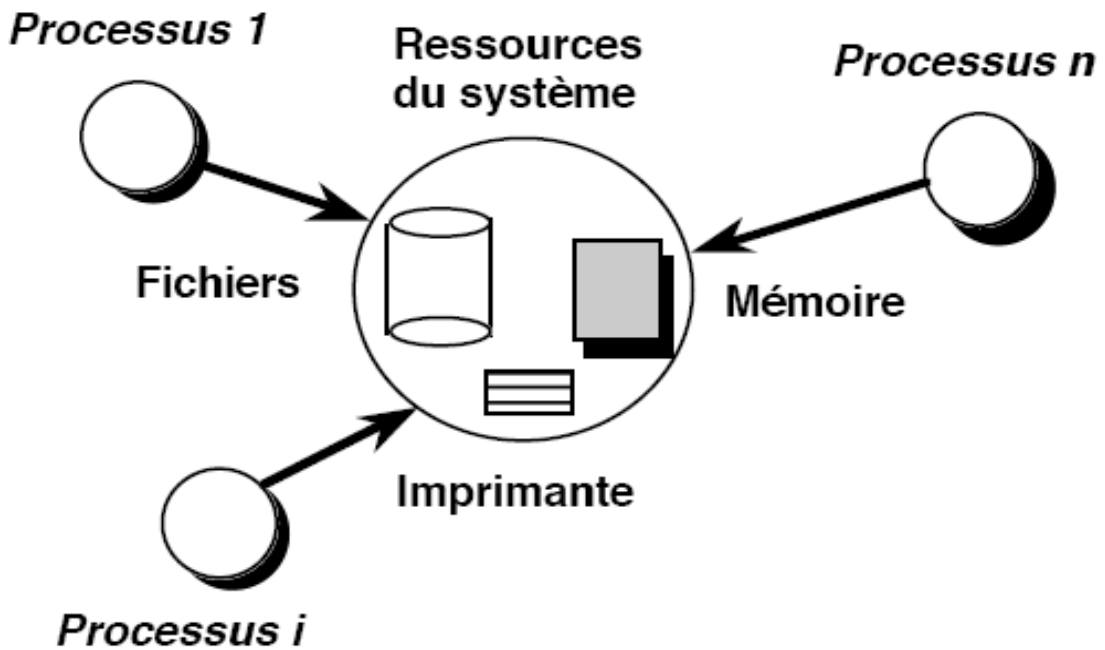
Une ressource se caractérise par son état et le nombre d'exemplaires (nombre d'utilisateurs maximal).

L'utilisation d'une ressource doit respecter un protocole en trois étapes :

- **Allocation** (synchronisation sur disponibilité d'un exemplaire de la ressource) : étape d'attente de la ressource (bloquante),
- **Utilisation** : étape d'occupation de la ressource,
- **Libération** (**Restitution**) de la ressource.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Ressource critique



Accès exclusif : une ressource critique est une ressource ne pouvant être utilisée que par un seul processus à la fois.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exclusion mutuelle

L'accès à une ressource critique doit se faire en **exclusion mutuelle**.

Pour réaliser l'exclusion mutuelle, il y a plusieurs conditions ou propriétés à respecter :

1. à tout instant **un processus au plus** peut se trouver en section critique,
2. si plusieurs processus sont bloqués en attente de la ressource critique, **l'un d'eux doit pouvoir y entrer au bout d'un temps fini**,
3. si un processus est hors de sa section critique, **il ne doit pas empêcher** l'entrée d'un autre processus dans sa section critique,
4. **la solution doit être la même** pour tous les processus.

Hypothèse :

- Tout processus sort de sa section critique au bout d'un temps fini.

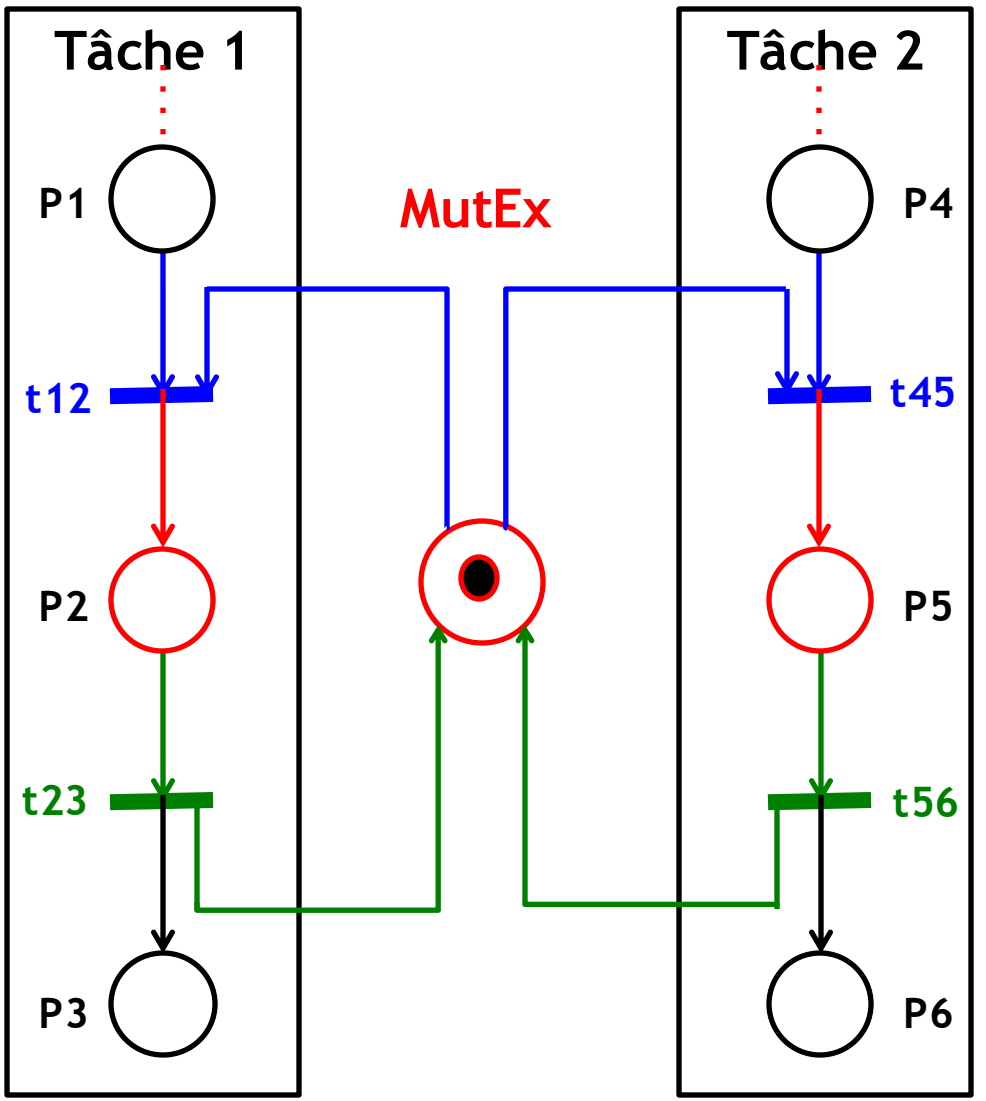
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exclusion mutuelle – modèle

La tâche 1 ne peut se trouver en P2 alors que la tâche 2 est en P5 et vice versa. Il y'a donc **exclusion mutuelle** entre P2 (**section critique** de la tâche 1) et P5 (section critique de la tâche 2). On parle d'exclusion mutuelle entre processus, chacun mettant en jeu une section critique (d'exclusion).

Ceci est symbolisé par une **ressource partagée** entre les processus avec un jeton **unique** (MuTex).

Ainsi, quand P2 est marquée, la MuTex est **indisponible** et donc aucun autre processus ne peut être dans sa section critique



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exclusion mutuelle – exemple

t0 (État initial) : P1 vient d'être marquée, la MutEx était **disponible** :

P1 & MutEx marquées.

t1 (tir de t12) : la tâche 1 rentre dans sa section critique et la MuTex devient **indisponible** :

P1 perd sa marque,

MuTex perd sa marque,

P2 est marquée.

t2 (marquage de P4) : la tâche 2 produit une marque dans P4 et se met en **attente** puisque la MutEx est indisponible.

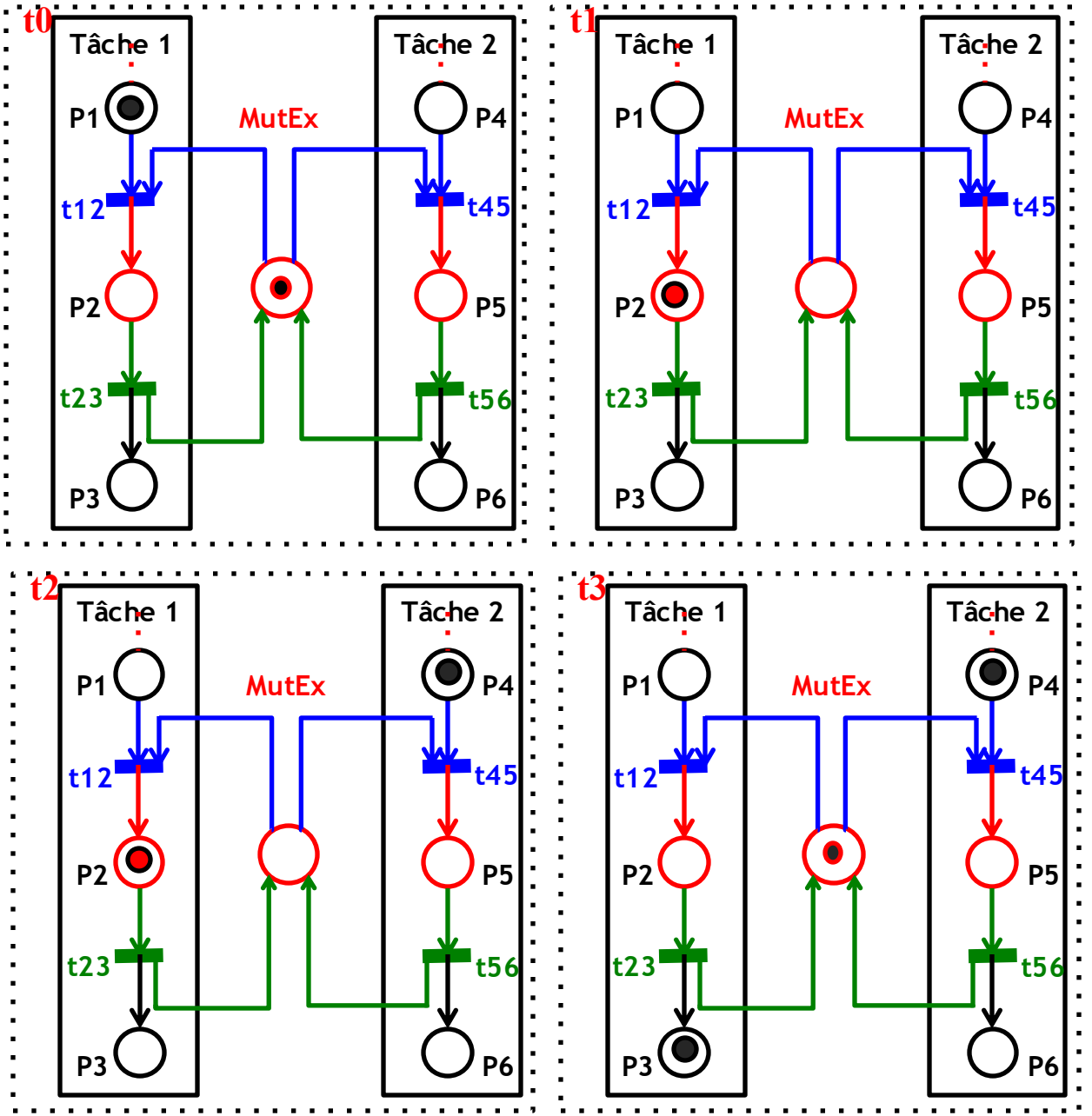
t3 (tir de t23) : la tâche 1 termine sa section critique et la MutEx devient **disponible** :

P2 perd sa marque,

MuTex est marquée,

P3 est marquée.

t4 : la tâche 2 peut rentrer dans sa section critique



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Implémentation – Désactivation des interruptions

L'**implémentation** la plus simple pour réaliser l'exclusion est de désactiver les interruptions durant le traitement critique : l'ordonnanceur ne peut donc interrompre le traitement.

Solution utilisée par le système en mode noyau :

- Le mode noyau permet la désactivation des interruptions
- Les programmes utilisateurs ne peuvent désactiver les interruptions :
  - Que se passerait-il si l'utilisateur ne réactive pas les interruptions après avoir quitté le traitement critique ?
  - Solution non appropriée
  - Il faut donc proposer un autre mécanisme.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Implémentation de la gestion des ressources

Plusieurs études ont été menées pour gérer une ressource critique (Dekker, Peterson).

Néanmoins, les solutions sont toutes basées sur l'attente **active** : consommation inutile des ressources CPU.

La solution qui a été la plus utilisée est à base de TSL (Test and Set Lock) : cellule câblée et ensuite instruction assembleur (TAS).

Les **sémaphores** de Dijkstra (1965) proposent un mécanisme utilisant une attente **passive** : réveil sur disponibilité de la ressource.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

TSL (Test And Set Lock) – TSL câblé

Programmation très complexe :

- utilisation en dernier ressort d'un mécanisme **câblé** qui réalise une forme élémentaire d'exclusion.

Il s'agit d'une instruction élémentaire TAS (Test And Set) sur une cellule mémoire. Le test et l'affectation forment donc une instruction atomique (indivisible).

```
void TSL(cellule m) {
                // Bloquer l'accès à la cellule m
                // Lire la valeur de la cellule m
                if (m==0) m=1;                  // allocation
                // Libérer l'accès à la cellule m
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

TSL soft – TAS

```
int TSL(int * verrou) {
        int temp;
        masquer_it;
        temp = *verrou;
        *verrou = 1;
        return temp;
        démasquer_it;
}
```

Utilisation : Mutex est initialisée à 0 (libre)

```
        while ( TSL(&MutEx) ) ;
        // Traitement section critique
        MutEx = 0;
```

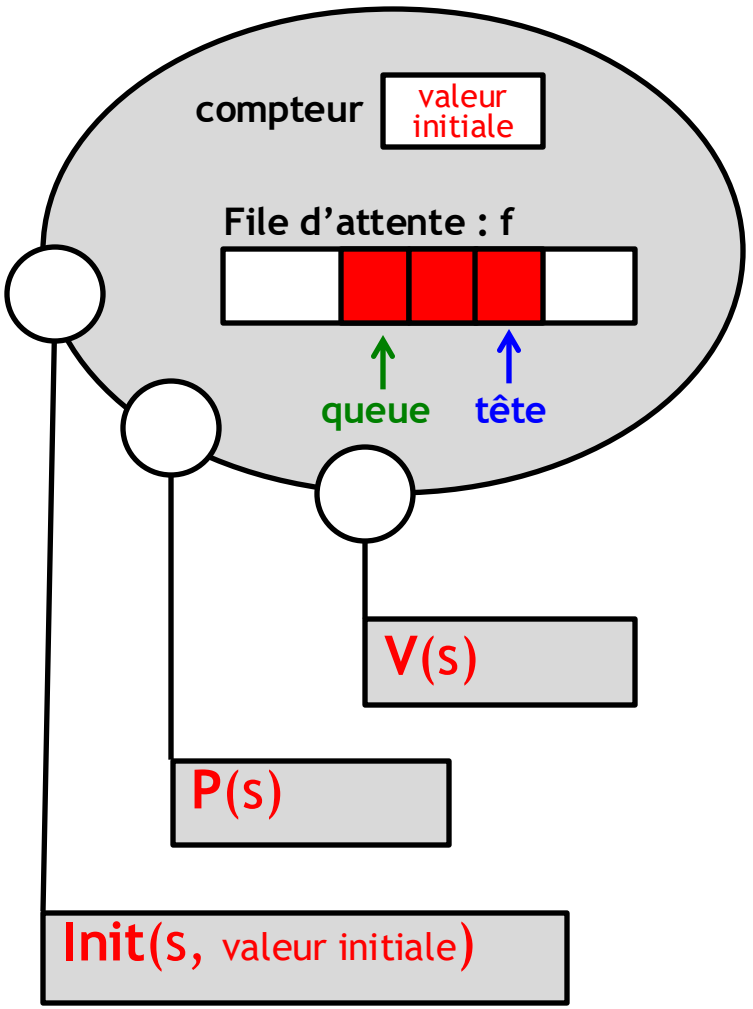
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

SÉMAPHORES

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille
IG2I

Les Sémaphores de Dijkstra



Les sémaphores de Dijkstra permettent la mise en œuvre de la **synchronisation** et le **partage de ressources** entre un nombre quelconque de processus (n>0).

Un sémaphore (**à compteur**) se caractérise par :

- une variable entière représentant le **nombre d'exemplaires disponible** :
  - une valeur **initiale,**
- une **file d'attente** des processus bloqués sur indisponibilité.

Un sémaphore est géré par trois primitives **indivisibles** (**interruptions masquées**) :

- **Init**(s, valeur initiale),
- **P**(s) : Allocation/Prise du sémaphore,
- **V**(s) : Libération/Restitution de sémaphore.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Les sémaphores – Initialisation

L'opération Init :

elle a pour but d'initialiser le sémaphore, c'est-à-dire qu'elle met à vide la file d'attente f(s) et **initialise le compteur avec la valeur initiale** fournie en paramètre. On définit ainsi le nombre de jetons initiaux dans le sémaphore.

```
Init(semaphore *s, int valInitiale)
{
    masquer_it;
    s➔compteur = valInitiale;
    // Vide la file d'attente s➔f ,
    // en mettant s➔tête et
    // s➔queue dans la position
    // initiale
    démasquer_it;
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Les sémaphores – Allocation

L'opération P(s) :

L'opération P(s) **attribue** un jeton au processus appelant s'il en reste au moins un ; sinon elle **bloque** le processus courant en le mettant en queue de la file f(s).
L'opération P est donc une opération **bloquante** pour le processus élu qui l'effectue en cas d'**indisponibilité**, Il y a donc ré-ordonnancement et un nouveau processus prêt est élu.
Concrètement, le compteur du sémaphore est **décrémenté** d'une unité. Si la valeur du compteur devient **négative**, le processus est bloqué.

```
P(semaphore s)
{
    masquer_it;
    s➔compteur--;
    if (s➔compteur<0) {
    // Enfiler le processus courant dans
    //  la file d'attente et le bloquer
        enfiler(getpid(), s➔f);
        Bloquer le processus courant;
    };
    démasquer_it;
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Les sémaphores – Libération

L'opération V(s) :

L'opération V(s) a pour but de déposer un jeton au sémaphore.

De plus, si il y a au moins un processus bloqué dans la file d'attente f(s) du sémaphore, le processus en tête de file est alors réveillé. La gestion des réveils s'effectue en mode FIFO.

L'opération V est une opération qui n'est jamais bloquante pour le processus qui l'effectue.

```
V(semaphore s)
{
    masquer_it;
    s➔compteur++;
    if (s➔compteur <= 0) {
    // il y a au moins un processus
    // en attente, défiler le
    // processus de tête et le
    // débloquer
        r = défiler (s➔f);
        Mettre r dans l'état ELU;
    };
    démasquer_it;
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Sémaphores POSIX

Un type opaque **sem_t** est défini dans **semaphore.h**.

Il existe deux types de sémaphores :

- Sémaphore **anonyme** (non-nommé), naturellement partagé entre threads, à condition d'être déclaré en variable globale :
  - Partage possible entre processus parents à condition de placer le sémaphore dans une mémoire partagée.
  - Implémentés par Linux à partir du noyau 2.6 et non implémenté par MacOS.
- Sémaphore **nommé** (mappage sur un fichier)
  - Partageable entre tous les processus : un sémaphore en mémoire partagée mappé sur un fichier (les droits d'accès sur les fichiers sont applicables).

Un sémaphore anonyme est initialisé par **sem_init()** alors qu'un sémaphore nommé est créé par **sem_open()**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Bibliothèque – sémaphores anonymes

Appel système	Rôle
#include <semaphore.h> // Définit le type sem_t	
int sem_init(sem_t *sem, int pshared, unsigned int value)	Initialisation du compteur de **sem** avec **value**. le paramètre **pshared** indique si le sémaphore sera partagé entre processus (le partage entre threads est implicite). Dans ce cas, il faut placer le sémaphore en mémoire partagée ou utiliser plutôt **sem_open()** pour créer un **sémaphore nommé**.
int sem_wait(sem_t *sem)	**Allocation** : décrémente le compteur de **sem**. Si compteur<0 alors **attente** jusqu'à ce qu'il devienne >=0. Appel **interruptible** par un **signal !**
int sem_post(semt_t *sem)	**Restitution** : incrémente le compteur de **sem**.
int sem_destroy(sem_t *sem)	Libération des ressources système associé au sémaphore **sem**.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Bibliothèque – sémaphores anonymes – suite

Appel système	Rôle
#include <semaphore.h> // Définit le type sem_t	
int sem_trywait(sem_t *sem)	Identique à sem_wait() mais non bloquant. Si compteur<=0 alors retourne -1 et errno vaut EAGAIN.
int sem_getvalue(sem_t *sem, int *sval)	Retourne la valeur du compteur de sem. Attention, la valeur fournie n'est plus la bonne au moment de son utilisation.

Exemple d'initialisation :

```
#include <semaphore.h>
sem_t mon_semaphore;

CHECK(sem_init(&mon_semaphore, 0, 1) ) , "—PB: sem_init()");
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Bibliothèque – sémaphores nommés …

Appel système	Rôle
#include <semaphore.h> #include <sys/fcntl.h>	
sem_t *sem_open(const char *name, int oflag)	Cette première forme utilise la valeur 0 pour oflag pour la distinguer de la 2° forme. Elle sert à **ouvrir** le fichier sémaphore (**préalablement créé**) désigné par name et retourne un pointeur de type sem_t sur le sémaphore (incrémente le nombre de liens). Si le fichier n'existe pas alors SEM_FAILED est retournée et errno vaut ENOENT.
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value)	La deuxième forme sert à **créer** le fichier sémaphore désigné par name, à l'**initialiser** avec value et à retourner un pointeur de type sem_t. Un fichier est créé dans /dev/shm/ sous le nom de sem_name, avec les droits d'accès fournies par mode. Les droits peuvent être spécifiés sous forme d'une **valeur octale** ou en utilisant les symboles définir par fcntl.h. Quand oflag vaut O_CREAT alors le fichier est créé s'il n'existe pas (mode et value sont alors ignorés). Quand oflag vaut O_CREAT|O_EXCL alors le fichier est créé à condition qu'il n'existe pas. Dans le cas contraire, SEM_FAILED est retournée et errno vaut EEXIST.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Bibliothèque – …sémaphores nommés

Appel système	Rôle
#include <semaphore.h> #include <sys/fcntl.h>	
int sem_wait(sem_t *sem)	**Idem** que pour les sémaphores **anonymes**
int sem_post(sem_t *sem)	
int sem_getvalue(sem_t *sem, int *sval)	
int sem_close(sem_t *sem)	Ferme le lien avec le fichier sémaphore (décrémente le nombre de liens).
Int sem_unlink(const char *name)	Supprime le fichier sémaphore désigné par **name**. Le sémaphore est détruit après fermeture de tous les processus utilisateurs (nombre de liens = 0).

Exemple de création :

```
#include <semaphore.h>
sem_t *mon_semaphore;
mon_semaphore=sem_open( "nom_du_semaphore", O_CREAT|O_EXCL , 0660, 1);
```

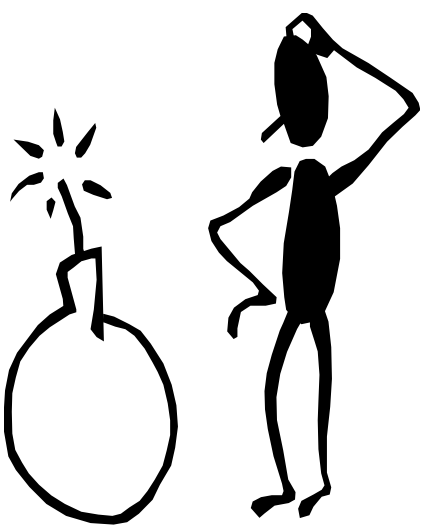
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Sémaphore et signal

Les primitives de gestion de sémaphore **s'arrêtent** sur réception d'un **signal** !!!

```
#include <semaphore.h>
#include <errno.h>

int sem_wait_restart(sem_t *sem) {
        int retval;
        while ( (retval = sem_wait(sem) ) == -1)
                && (errno == EINTR))
        ; // on relance le sem_wait()
        return retval;
}
```



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Threads – Partage de données – pthread_mutex_t

Le multithreading permet le partage des variables globales (statiques & allouées dynamiquement) mais il faut protéger l'**écriture** par un **accès exclusif**.

Ne peut être restitué que par le thread utilisateur.

Déclaration d'un thread de type mutex :

```
static pthread_mutex_t    mutex;
```

Utilisation de la mutex :

```
pthread_mutex_init(&mutex, NULL);
pthread_mutex_lock(&mutex);
pthread_mutex_unlock(&mutex);
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

INTERBLOCAGE

Problème des philosophes

Modèle Producteur(s) / Consommateur(s)

Problème du coiffeur endormi (non traité)

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille IG2I

Problème des philosophes

Un philosophe passe son temps à manger et à penser.

Lorsqu'un philosophe a faim, il tente de s'emparer des deux fourchettes qui sont à sa droite et à sa gauche, une après l'autre. L'ordre importe peu. S'il obtient les deux fourchettes, il mange pendant un certain temps, puis repose les fourchettes et se remet à penser (pendant un temps long que celui qu'il a passé à manger).

L'objectif est d'écrire un programme simulant la tablée où chaque philosophe se livre à ses activités (penser & manger) sans jamais être bloqué ou bloquer un autre philosophe.



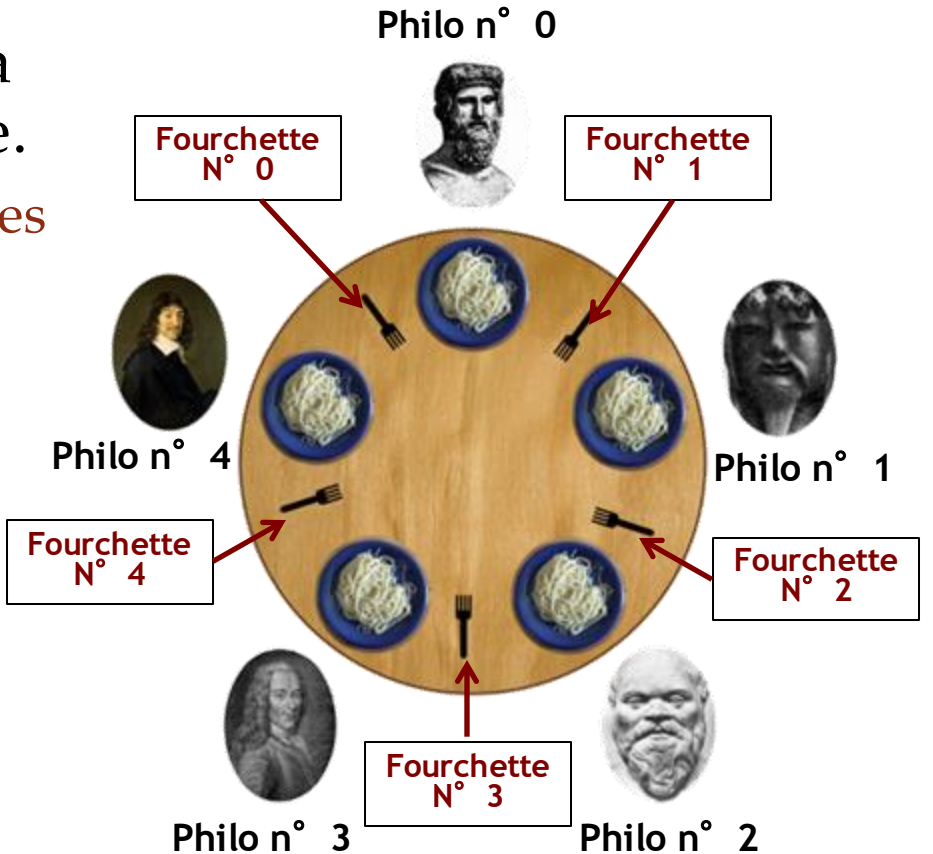
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Philosophes – analyse

Chaque philosophe partage la fourchette à sa gauche avec son voisin de gauche et la fourchette à sa droite avec son voisin de droite.

- Gestion circulaire (table ronde) des indices.
- 0 à NB_PHILOS

```
int vd(int i) {
  if (i==0) return NB_PHILOS-1;
  return i-1;
}
int vg(int i) {
  if (i==NB_PHILOS-1) return 0;
  return i+1;
}
```



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Philosophes – modèle RdP – solution intuitive

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Les philosophes : 1° solution

```
void Philosophe (int i) {
      Penser;    //le philosophe pense
      //Prendre fourchette gauche
      Prendre_fourchette(i);
      //Prendre fourchette droite
      Prendre_fourchette(i+1);
      Manger ;   // miam, miam, spaghettis
      //Poser fourchette gauche
      Poser_fourchette(i);
      //poser fourchette droite
      Poser_fourchette(i+1);
}
```



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Philosophes – modèle RdP – solution par synchro

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Les philosophes : 2° solution

```
semaphore SP[n];      // tableau de sémaphores privés
etat e[n];     // tableau d'états des philosophes
void Philosophe (int i) {
     Penser;   //le philosophe pense
     //Action pour demander de manger
     P(mutex)
          if (e[i-1]!=M && e[i+1]!=M) {e[i]=M; V(SP[i]);}
          else e[i] = S;
     V(mutex)
     P(SP[i])
     Manger ;  // miam, miam, spaghettis
     // Action pour arrêter de manger
     P(mutex)
          e(i) = P;
          if (e[i+1]==S && e[i+2]!=M) {e[i+1]=M; V(SP[i+1]);}
          if (e[i-1]==S && e[i-2]!=M) {e[i-1]=M; V(SP[i-1]);}
     V(mutex)
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Philosophes – généralisation de la solution par synchronisation

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Les philosophes – Solution générale …

```
semaphore SP[n];//tableau de sémaphores privés v.i.=0
semaphore mutex;//v.i.=1
etat e[n];              //tableau d'états v.i.=P

// il faut définir deux macros :
// vd(i) pour voisin droit
// et vg(i) pour voisin gauche

void tester (int num) {
    if (e[num]==S
        && e[vd(num)]!=M
        && e[vg(num)]!=M) {
           e[num] = M;
           V(SP[num]);
    }
}
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

… Les philosophes – Solution générale

```
void Philosophe (int i) {
    Penser();        //le philosophe pense
    //Action pour demander de manger
    P(mutex);
        e[i] = S;
        tester(i);
    V(mutex)
    P(SP[i])
    Manger() ;       // miam, miam, spaghettis
    // Action pour arrêter de manger
    P(mutex);
        e(i) = P;
        tester(vd(i));
        tester(vg(i));
    V(mutex);
}
```

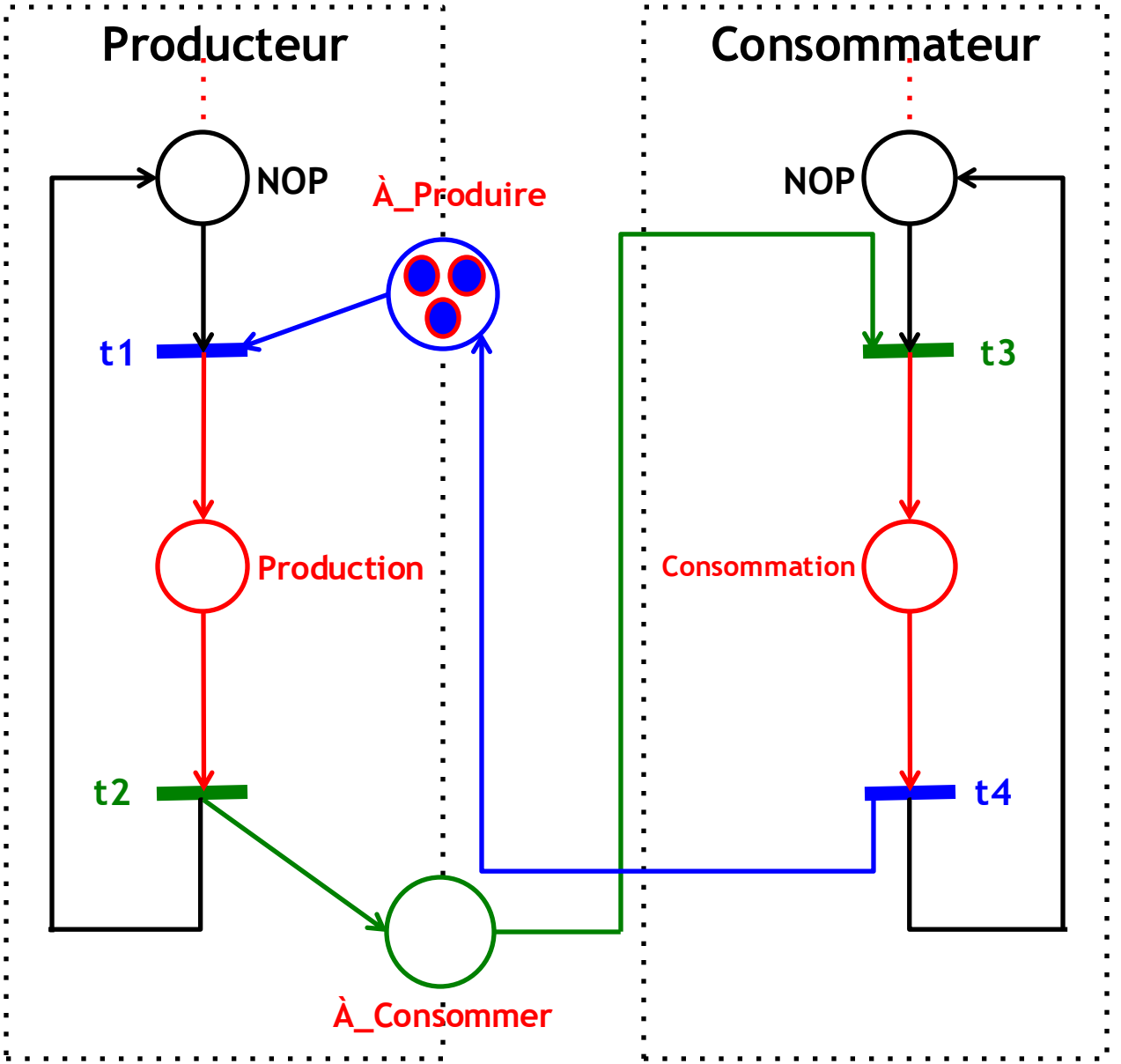
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Producteur/Consommateur – 1P/1C

Bloquer le producteur sur manque de jetons À_Produire (VIDE), de même bloquer le consommateur sur indisponibilité de jetons À_Consommer (PLEIN).

Un jeton représente 1 unité matérielle qui ne peut être consommée qu'une seule fois.

Une variante de ce modèle est le rédacteur/lecteur : on produit de l'information qui a une consommation multiple.



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Rédacteur/Lecteur – 1R/1L

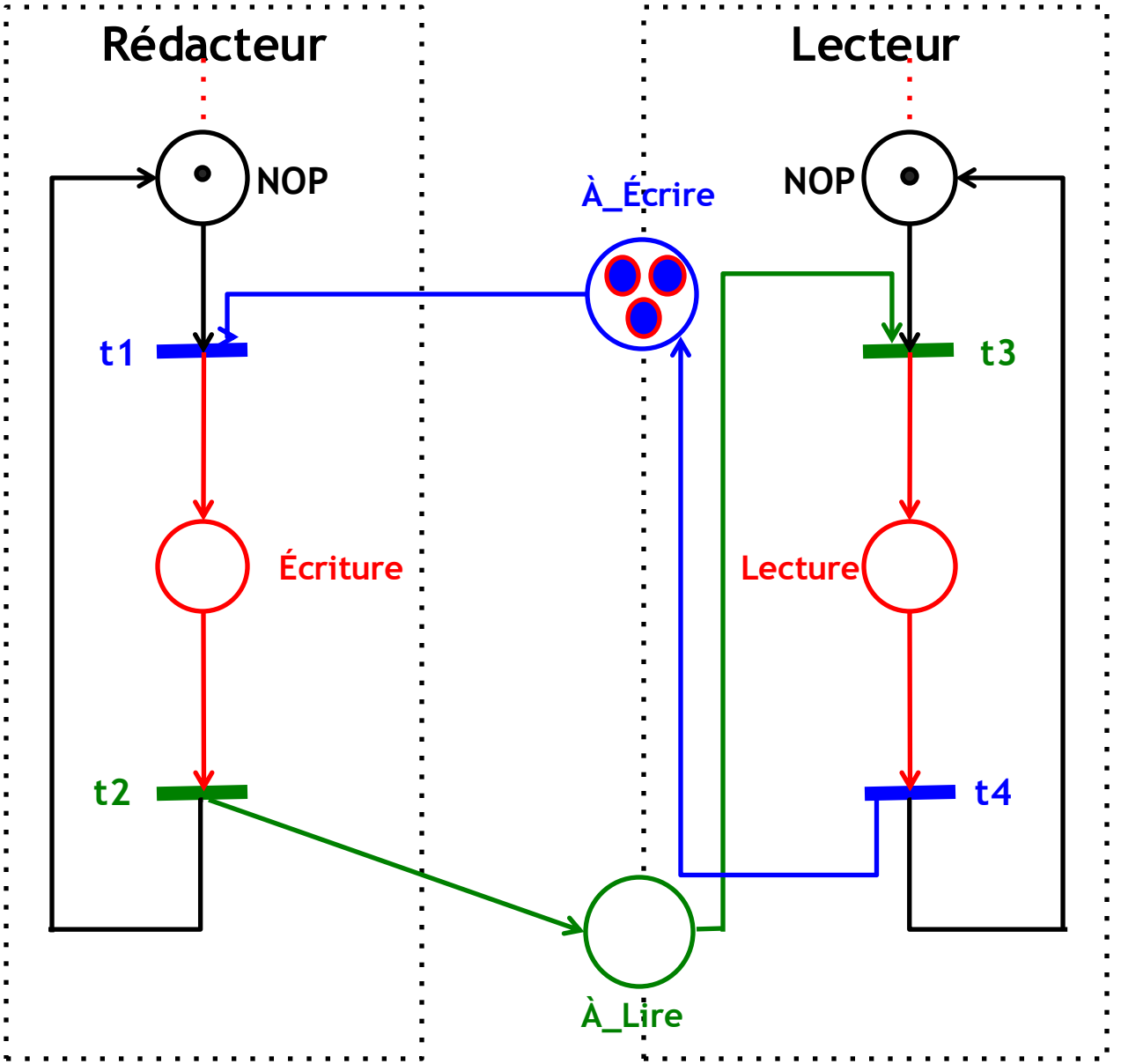
Un jeton représente 1 unité informationnelle qui ne peut être consommée plusieurs fois.

Un exemple d'implémentation sous UNIX : le pipe :

$ processus1 | processus2

On exécute 2 processus avec un tampon mémoire (fichier UNIX) en écriture pour le premier et en lecture pour le 2° .

Blocage du 2° processus sur indisponibilité de données et relance d'écriture après lecture.



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Rédacteur / Lecteur

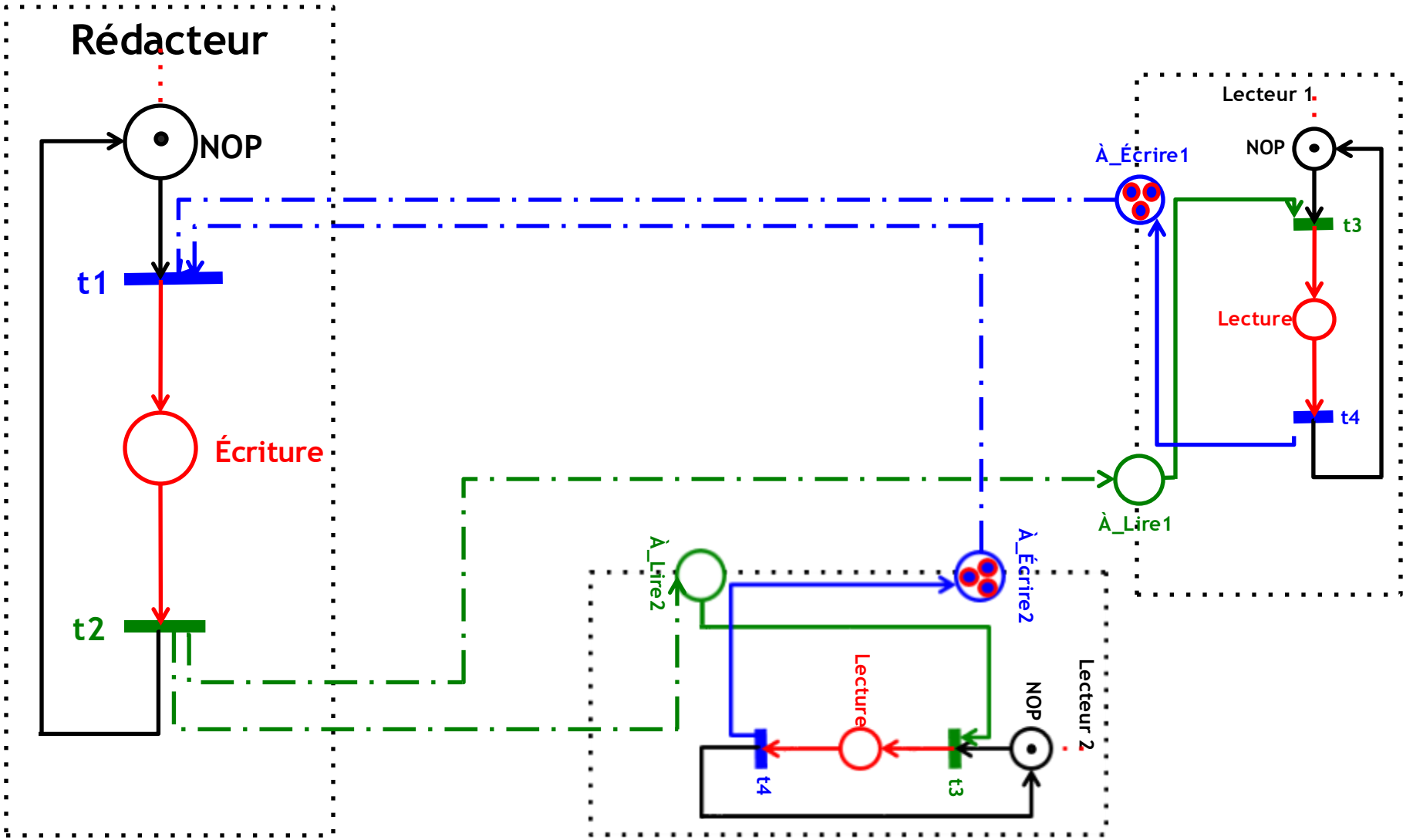
Rédaceur()	Lecteur()
Produire le message	
`VIDE --;` `si VIDE == -1 alors attendre`	`PLEIN --;` `si PLEIN == -1 alors attendre`
Déposer un message	***Prélever un message***

`PLEIN ++;`
`si lecteur en attente`
`alors le réveiller`
	`VIDE ++;`
	`si rédacteur en attente`
	`alors le réveiller`

	Consommer le message
Recommencer	***Recommencer***

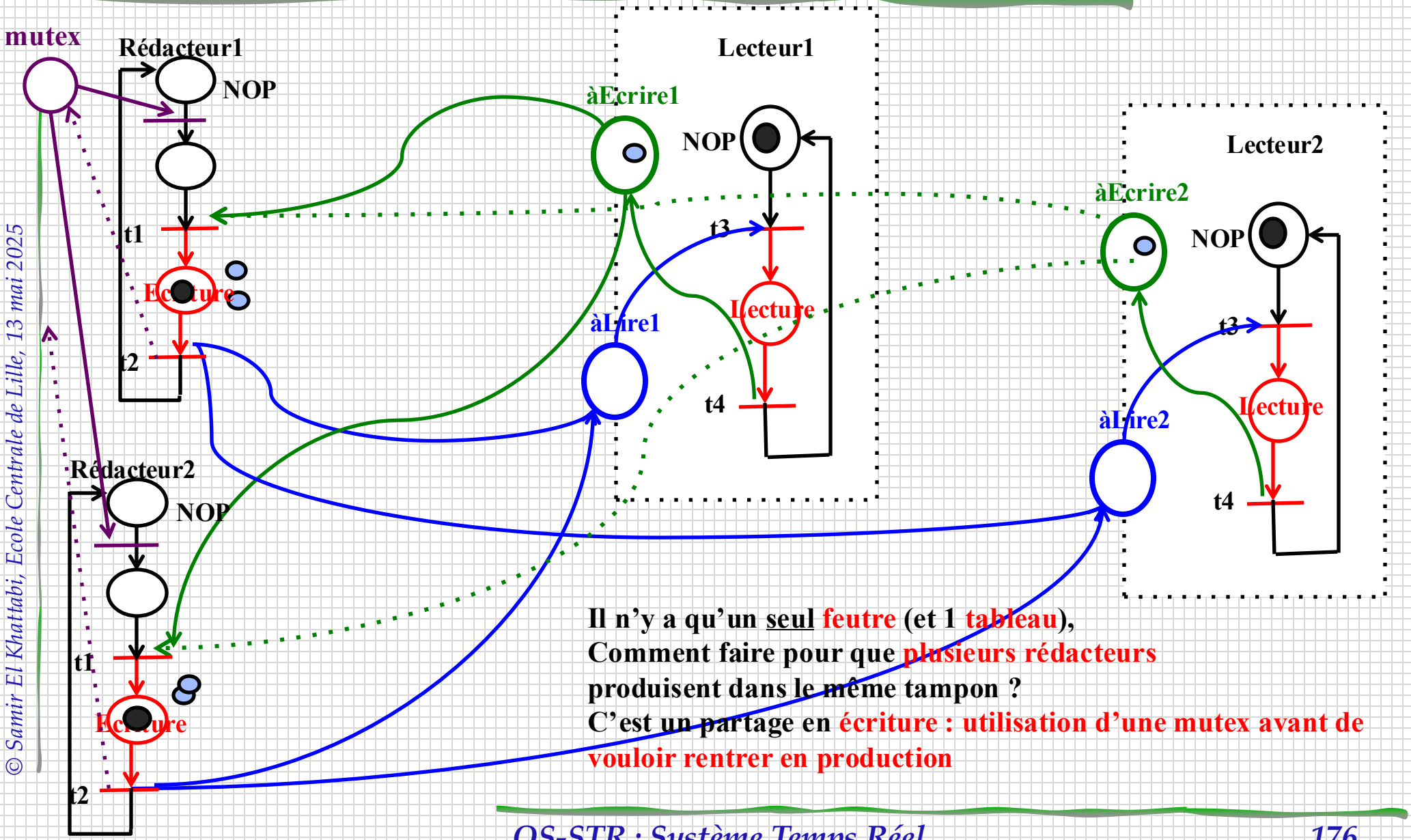
© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

1R/mL – Plusieurs lecteurs



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

nR/mL – plusieurs Rédacteurs



Il n'y a qu'un seul feutre (et 1 tableau),
Comment faire pour que plusieurs rédacteurs produisent dans le même tampon ?
C'est un partage en écriture : utilisation d'une mutex avant de vouloir rentrer en production

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Interblocage

L'interblocage est dû à l'imbrication de ressources :

- Considérons à présent la situation suivante pour laquelle deux processus P1 et P2 utilisent tous les deux, deux ressources critiques R1 et R2 selon le code suivant.

Processus 0	Processus 1

P(R1);
P(R2);
// Utilisation de R1 & R2
V(R2);
V(R1);
	P(R2);
	P(R1);
	// Utilisation de R1 & R2
	V(R1);
	V(R2);

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Interblocage – conditions de réalisation

4 conditions pour être en interblocage :

- Une ressource au moins est non partageable
- Occupation d'une ressource et attente d'une autre ressource
- Pas de réquisition : la libération d'une ressource ne peut se faire que par le détenteur
- Attente circulaire : il existe un **cycle d'attente** entre au moins deux processus.

Exemple d'attente circulaire :

- 2 processus et 3 ressources
- P1 : R1, R2, R3
- P2 : R3, R2, R1
- Scénario de blocage : P1/R1, P2/R3, P1/R2,
- P1 attend R3 et P2 attend R1

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Interblocage – traitement

4 méthodes de traitement :

- Détection/Guérison : algorithme basé sur un graphe représentant l'allocation des ressources et les processus en attente
- Prévention :
  - Éviter la circularité en imposant un ordre total.
  - Protéger la prise de ressources par une mutex.
- Evitement : à chaque demande d'allocation, un algorithme de sûreté est déroulé pour prévenir un interblocage à venir. Si tel est le cas, l'allocation est différée. Cette technique est basée sur la notion d'état sain. Ainsi, si l'état est malsain, on préfère ne pas attribuer la ressource : algorithme du banquier
- Autruche

Les 3 premières politiques sont très coûteuses alors on applique souvent la politique de l'autruche

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Coiffeur endormi

Cadre : Salon avec un fauteuil et N chaises.

Quand il n'y a pas de clients, notre coiffeur s'endort dans le fauteuil (chose qu'il commence par faire dès son arrivée au matin).

Le client doit réveiller notre coiffeur à son arrivée. S'il y a des clients et qu'il y a une chaise vide alors il attend son tour (éveillé).

Si le salon est plein, le client repart sans se faire coiffer.

Proposez une solution sans produire aucune situation de concurrence.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

MÉMOIRE PARTAGÉE

Bibliothèque POSIX pour le partage mémoire

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille
IG2I

Shared Memory

Partage de l'espace d'adressage en vue d'une IPC (Communication InterProcess) optimale.

Cette solution est utile pour des processus ayant des droits d'accès différents :

- Processus appartenant à des utilisateurs différents
- Pour une application appartenant à un même utilisateur, il faut privilégier les threads.

Approche de mise en œuvre :

- On utilisation un fichier «mappé et partagé»,
- Les données sont écrites dans le fichier par l'intermédiaire de l'espace d'adressage réservé, sont accessibles à l'ensemble des processus mappés sur ce fichier.
- La protection repose sur les droits d'accès applicables aux fichiers.
- Ce mécanisme repose sur les sémaphores.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Mise en œuvre

Le protocole d'accès à une zone mémoire partagée se fait en plusieurs étapes :

- Ouverture (avec création éventuelle) d'un objet mémoire partagé : pseudo-fichier. Cette opération crée un lien sur le fichier.
- Allocation et mappage d'une zone mémoire sur l'objet mémoire
- Exploitation de la zone mémoire en partage
- Démappage
- Clôture
- Suppression du lien : quand le nombre de liens est à 1, l'objet mémoire est détruit.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Fonctions de base

Ouverture : **shm_open()**

- Ouvre un objet mémoire partagé et retourne un descripteur de fichier

Etalonnage : **ftruncate()**

- Taille le fichier (objet mémoire) en un multiple de pages mémoire

Mappage : **mmap()**

- Mappe l'objet en mémoire

Déconnexion : **munmap()**

- Démappe l'objet en mémoire

Fermeture : **shm_close()**

Suppression : **shm_unlink()**

- Détruit l'accès au fichier

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Ouverture

int shm_open(const char *name, int oflag, mode_t mode);

- name : nom du fichier mémoire (dans **/dev/shm**)
- oflag : (comme pour sémaphore)
  - ou logique entre **O_RDONLY**, **O_RDWR**, **O_CREAT**, **O_EXCL** (exclude), ...
- mode : seulement si O_CREAT
  - syntaxe dans /usr/include/sys/mode.h

```
#include <sys/mman.h>
#include <sys/mode.h>
int fd;                 // file descriptor
fd = shm_open("maZone",
              O_CREAT|O_RDWR,
              S_IRWXU)
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Mappage

caddr_t mmap(caddr_t addr, size_t len, int prot, int flags, int filedes, off_t off)

- addr : adresse de début de la région (0 : attribution par le système)
- len : taille de la région mappé en multiple de pages
- prot : **PROT_NONE**, **PROT_READ**, **PROT_WRITE**, **PROT_EXEC**
- flags:
  - **MAP_SHARED** partage des modifications
  - **MAP_PRIVATE** modifications privées
  - **MAP_FIXED** interprète rigoureusement les adresses !
- filedes : file descriptor (créé par **shm_open()**)
- off : offset dans le fichier

```
pageSize = sysconf(_SC_PAGE_SIZE);
CHECK(ftruncate(fd, pageSize),  "Problème ftruncate");
virtAddr = mmap(0, pageSize, PROT_WRITE, MAP_SHARED, fd, 0);
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Utilisation

On utilise l'adresse de la région mappée comme un tableau d'octets :

```
virtAddr[index_dans_l_objet_mémoire_partagé]
```

Exemple :

```
virtAddr[i]='a';
```

int msync(caddr_t addr, size_t len, int flags);

addr : **virtAddr**

len : **pageSize**

flags :

MS_SYNC cohérence synchrone

MS_ASYNC les maj sont planifiées

MS_INVALIDATE oblige nouvelle consultation (cache)

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Clôture

Il est nécessaire d'effectuer les opérations nécessaires à la libération des espaces mémoires et la clôture des objets mémoires :

int munmap(caddr_t addr, size_t len);

int shm_close(int filedes);

int shm_unlink(const char *name);

- comme pour sémaphore

```
munmap(virtAddr, pageSize);
close(fd);
shm_unlink("maZone");
```

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exercices didactiques

Tester la mise en œuvre d'une zone commune avec un processus écrivain et un processus lecteur.



Utiliser le partage en mémoire pour implémenter le problème des philosophes et spaghettis : on partage l'état des philosophes.

Mettre en œuvre l'algorithme du producteur/consommateur en utilisant la zone mémoire comme une FIFO de messages.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

ANNEXE :
SYSTÈME DE FICHIERS

également la banalisation des E/S en fichiers

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille
IG2I

Système de fichiers

Un système de fichier est une structuration hiérarchique (répertoire/fichier) d'un espace disque :

- Découpage de l'espace en deux parties :
  - Espace de gestion des index (moyen d'accès aux données)
  - Espace de stockage des données
- Appels système pour créer/accéder, lire/écrire, …

Comment gérer plusieurs disques appartenant à une même machine ?

- Une première solution consiste à installer un système de fichiers sur chaque disque et de les gérer séparément (cas du DOS, VMS & NT).
- Une deuxième solution consiste à **monter** l'arborescence d'un disque sur une autre arborescence de disque (cas d'UNIX). Différents systèmes de fichiers cohabitent dans la même arborescence.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Accès concurrents à un fichier

Utiliser un sémaphore sur le fichier : trop contraignant

Verrouiller une zone d'un fichier : deux processus appartenant à des utilisateurs différents peuvent alors lire en même temps deux zones différentes d'un même fichier.

Le verrou peut être soit **exclusif** ou **partagé** (seuls les appels partagés seront acceptés).

Un appelant peut rester bloqué (s'il le souhaite) tant que le verrou n'est pas levé.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Fichiers & Arborescence (1)

Un fichier UNIX est une suite d'octets au niveau le plus bas (taille quelconque) : banalisation de la notion de fichier.

Nom de fichier :

- Aucune restriction sur les noms de fichiers (N'importe quel caractère).
- Les fichiers commeçcant par un point servent à la configuration de l'environnement.

La notion de répertoire (un fichier structuré en une suite de couples fichier/adresse) permet d'introduire une hiérarchie entre les fichiers :

- Regroupement de fichiers et de répertoires, concernant une même entité (personne, projet, application, …) dans un même répertoire.

Chemin d'accès à un fichier :

- Absolu : chemin complètement spécifié depuis la racine (/).

  Exemple : **/home/eleves/truc/.cshrc**
- Relatif : chemin spécifié depuis le répertoire courant.

  Exemple : **bin/exemple**

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Fichiers & Arborescence (2)

Répertoires particuliers :

- . & ..
- **Working directory** : répertoire de travail ou répertoire courant.
- **Home directory** : à chaque utilisateur correspond un répertoire privé qui est son répertoire de travail par défaut : il est automatiquement positionné sur ce répertoire lors de la connexion.

Classification des fichiers selon leur structure :

- **régulier** (-) : fichier texte ou exécutable
- **répertoire** (d) : un fichier contenant d'autres fichiers,
- **device** (c/b) : imprimante, terminal, disque dur, floppy, lecteur de bande, disque numérique, …
- **lien** (l)
- **socket** (s)
- **fifo** (p)
- **rien** : le trou noir (**/dev/null**)

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Répertoires d'un système UNIX

/bin	Commandes utilisateur de base
/sbin	Commandes d'administration
/dev	Fichiers spéciaux pour les E/S sur périphériques
/etc	Configuration et administration système
/lib	Bibliothèques système
/include	Fichiers d'entête système
/var	Journaux, Comptabilité & Queues
/tmp	Fichiers temporaires
/usr ou Extension du système	/usr/bin : Exécutables /usr/lib : Librairies /usr/include : Fichiers entête système /usr/man : Manuels d'aide en ligne /usr/var : Comptabilité & Queues
/home	Répertoires utilisateurs
/opt ou /usr/local	Applications spécifiques

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Implémentation du système de fichiers

Tous les disques UNIX possèdent la même structure (représentation sous forme d'un tableau):

- Bloc 0 : contient le code d'initialisation de l'ordinateur (non utilisé);
- Bloc 1 (**superbloc**) :
  - Nombre d'i-nodes
  - Nombre de blocs de données,
  - Pointeur sur la liste des blocs libres.
- La table des **i-nodes** : un ensemble contigu de blocs numérotés :
  - I-node 1 : liste des blocs défectueux
  - I-node 2 : racine du disque
- Les **blocs de données** : espace restant utilisé pour le stockage du contenu des fichiers et des répertoires. Les blocs d'un fichier sont non contigus en général.

La structure **i-node** permet le contrôle et la localisation des données pour un fichier :

- Informations de contrôle : mode, nb-liens, UID, GID, taille, Dates
- Informations de localisation : liste des dix premiers blocs, adresse du bloc d'indirection simple (double et triple).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Implémentation du système de fichiers (2)

Un répertoire est une suite non triée d'entrées de 16 octets (System V) où chaque entrée est une paire d'informations :

- Numéro d'i-node : 2 octets
- Nom de fichier : 14 octets

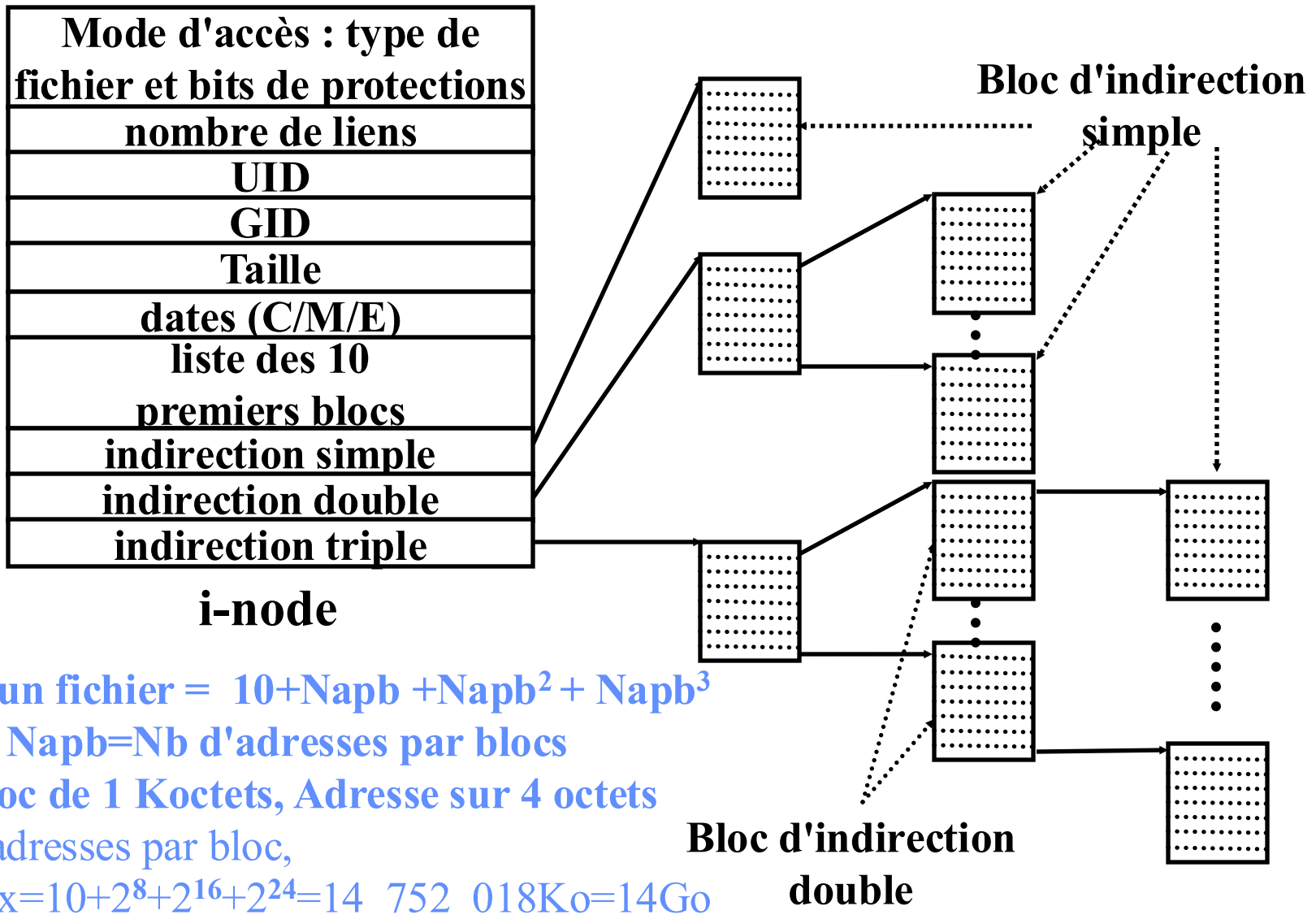
Par des comparaisons successives des répertoires composant un chemin d'accès à un fichier avec les entrées d'un répertoire, on finit par trouver le fichier (son i-node) ou non : le chemin relatif est plus rapide.

L'i-node est utilisé comme index dans la table des i-nodes sur disque.

La structure de l'i-node est chargée en mémoire dans la table des fichiers ouverts et pointée par la structure utilisateur du processus.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Implémentation d'un i-node



Taille max d'un fichier = $10+Napb+Napb^2+Napb^3$
avec Napb=Nb d'adresses par blocs

Exemple : Bloc de 1 Koctets, Adresse sur 4 octets

256 (2^8) adresses par bloc,

Taille max=$10+2^8+2^{16}+2^{24}$=14_752_018Ko=14Go

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Processus & Fichiers

L'appel système open(<nom de fichier>, <mode d'accès souhaité>) permet de tester **l'existence** du fichier et de **contrôler la cohérence** du mode d'accès avec les permissions du fichier spécifié.

Cet appel renvoie un entier positif (descripteur de fichier, à partir de 3) puisque par défaut tout programme possède trois descripteurs (0 pour STDIN, 1 pour STDOUT, et 2 pour STDERR).

Deux modes d'accès sont possibles pour les fichiers :

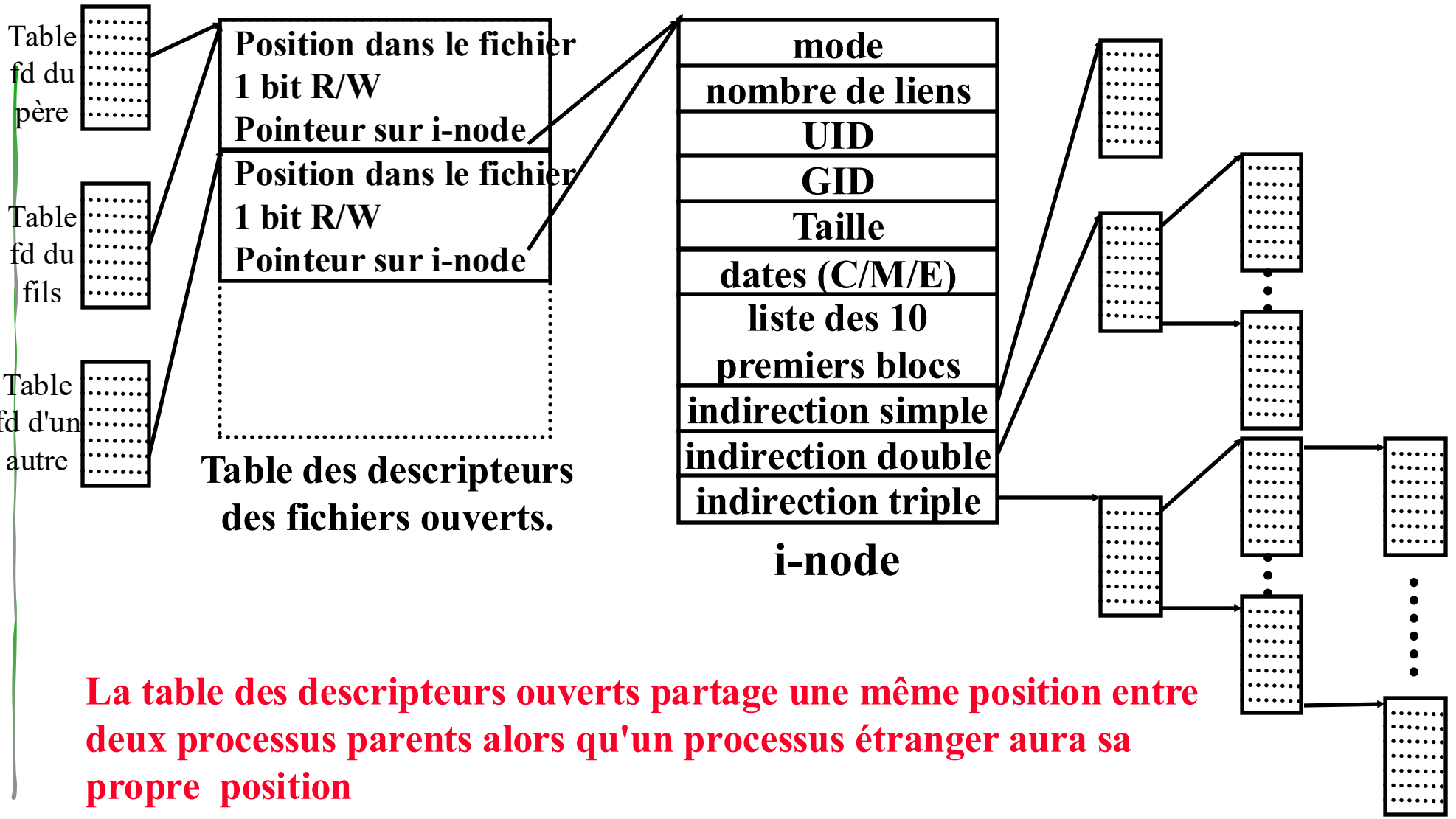
- Chemin absolu,
- Chemin relatif : variable d'environnement PWD (ou cwd) dans segment pile.

L'appel système link() permet à des utilisateurs de partager un même fichier.

L'appel système unlink() coupe le lien et supprime l'i-node si le nombre de liens est à 1.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Exemple d'un appel de lecture sur fichier



La table des descripteurs ouverts partage une même position entre deux processus parents alors qu'un processus étranger aura sa propre position

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Améliorations du système de fichiers de Berkeley

Les noms de fichiers sont stockés sur 255 caractères.

POSIX fournit les appels **OPENDIR**, **READDIR**, **CLOSEDIR** afin de rendre les programmes compatibles sur les deux systèmes.

La deuxième modification concerne la division d'un disque en un groupe de cylindres appelé partition ou disque logique. Un disque logique dispose de la même structure qu'un disque physique : superbloc, table d'i-nodes et blocs de données.

La troisième amélioration se situe au niveau de la taille bloc : en effet, elle peut être différente d'une partition à une autre. Il est ainsi possible d'avoir des partitions avec un bloc de 512 octets pour les petits fichiers et des partitions avec un bloc de 4 Ko pour les grands fichiers

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Principe d'Entrées / Sorties

La gestion des périphériques est intégrée dans le systèmes de fichiers en considérant les unités comme des fichiers spéciaux.

A chaque unité est associé un fichier dans **/dev** :

- **/dev/ttyxx** : terminal virtuel n° xx
- **/dev/lp** : port parallèle
- **/dev/hdx**: disque dur n° x (a, b, c, ….)
- **/dev/sdx** : disque SCSI n° x (a, b, c, ….)

Accessibilité similaire aux fichiers : on utilise les appels **READ** et **WRITE** comme on le ferait avec n'importe quel fichier.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Principe d'Entrées / Sorties (2)

Les mécanismes de protection habituels sont appliqués.

Deux types de fichiers spéciaux :

- Fichiers bloc destinés aux périphériques de stockage : adressage d'un bloc (lire/écrire un bloc)
- Fichiers caractère destinés au périphériques de flux : écran, réseau, clavier, ….

Les fichiers de type caractère ne peuvent pas utiliser l'adressage direct d'un caractère.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

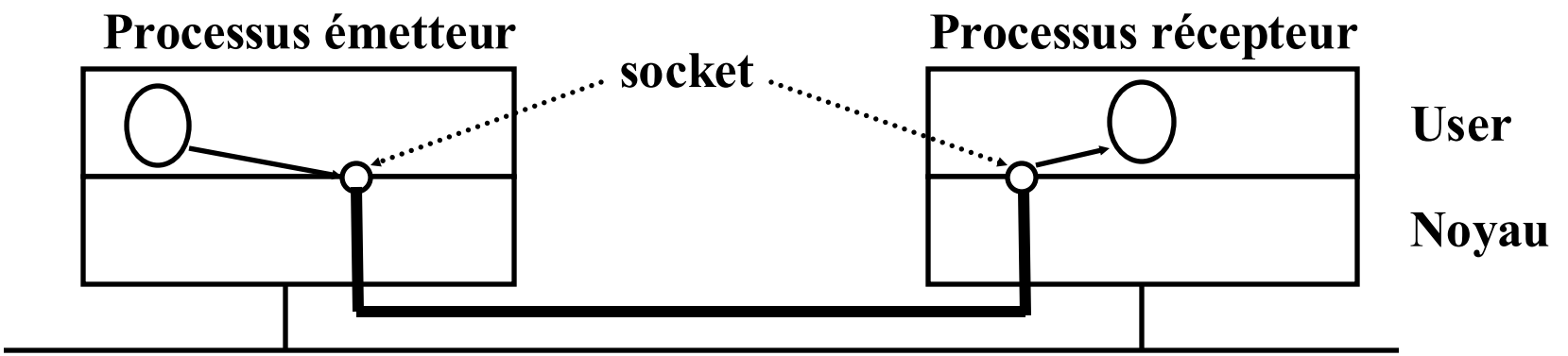
Principe d'Entrées / Sorties (3)

Les périphériques de type caractère disposent de caractéristiques spécifiques :

- Clavier : Effacement = DEL/BackSpace, …
- Ecran : Nombre de lignes, nombre de colonnes, …
- Réseau : Taille d'une trame, …

Il existe pour chaque type de périphérique, un ensemble d'appels systèmes permettant de modifier ces valeurs.

Un autre modèle d'IPC que les tubes est introduit par Berkeley : notion de socket (cf cours programmation réseaux TCP/IP).



© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

PERMISSIONS

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

centralelille
IG2I

Permissions (1)

Bases :

- Un utilisateur est membre d'un seul groupe.
- Un fichier appartient à un seul utilisateur et à un seul groupe.

Droits d'accès sur les fichiers :

- 3 permissions **rwx** applicables à 3 classes d'utilisateurs
  - **r** droit de lecture du fichier,
  - **w** droit d'écriture du fichier,
  - **x** droit d'exécution
- Les classes d'utilisateurs sont :
  - **user** le propriétaire du fichier
  - **group** les utilisateurs du même groupe que l'utilisateur,
  - **other** tous les autres utilisateurs.

Droits d'accès sur les répertoires :

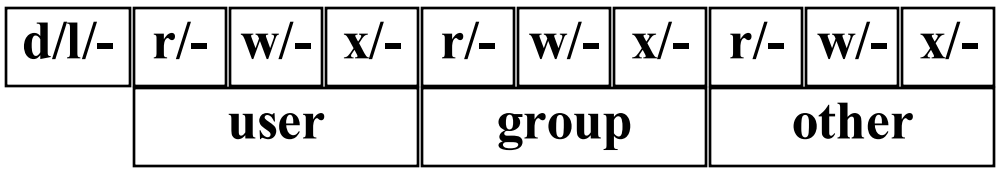
- Les clés **rwx** perdent leur signification habituelle :
  - **r** droit de lecture et donc de listage (**ls**),
  - **w** droit d'écriture sur le répertoire (**rm**),
  - **x** droit de traversée du répertoire (**cd**).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Permissions(2)

Expression des permissions :

- Les permissions d'un fichier (répertoire) sont exprimés sous la forme de 10 caractères :



d : répertoire **c** : device **b** : disk **-** : fichier régulier **l** : lien

r/- : read/not read **w/-** : write/not write **x/-** : execute/not execute

Protégez vos données :

- Utilisez **-rw-------** pour les fichiers,
- Utilisez **drwx--x--x** pour les répertoires.
- La commande **umask 066** permet de créer les fichiers et les répertoires avec les protections ci-dessus : sécurité et souplesse.
- La valeur de umask est exprimée en base **octale**.
- Pour obtenir les protections, on **XORe** 666 pour les fichiers et 777 pour les répertoires.

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Permissions(3)

Table XOR :

A	B	XOR
0	0	0
0	**1**	**1**
1	**0**	**1**
1	1	0

Expression des permissions en base octale :

- 3 permissions d'accès (**rwx** : 1 chiffre en base 8),
  0 : non autorisé **1** : autorisé,
- 3 classes d'utilisateurs (3 chiffres en base 8),
  Exemple :**-rwxrw-r--** correspond à **764**

La commande newgrp :

- Elle permet à l'utilisateur de changer de groupe et d'accéder ainsi aux fichiers et répertoires appartenant à des utilisateurs de ce groupe.
- Syntaxe : **newgrp groupe** (l'utilisateur doit appartenir au groupe).

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025

Permissions(4)

La commande **chown** :

- Cette commande permet de changer le propriétaire et éventuellement le groupe du fichier.

chown <user>[:<group>] <liste des fichiers à céder>

La commande **chgrp** :

- Cette commande permet de changer le groupe du fichier.

chgrp <group> <liste des fichiers à céder>

La commande chmod :

- Cette commande permet de changer les protections des fichiers.

chmod <perms en octal> <liste des fichiers à modifier>

chmod <liste des perms> <liste des fichiers à modifier>

Où chaque permission s'exprime comme suit :

[ugoa][+-][rwxst] ou [ugoa]=[une combinaison de rwxst]

© Samir El Khattabi, Ecole Centrale de Lille, 13 mai 2025